

ĐẠI HỌC CÔNG THƯƠNG THÀNH PHỐ HỒ CHÍ MINH
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO ĐỒ ÁN

Đề tài:

XÂY DỰNG ỨNG DỤNG QUẢN LÝ CHI TIÊU

Giảng viên: Đỗ Gia Bảo

Thành viên nhóm:

Trần Duy Bảo - 2001220330

Nguyễn Võ Như Ý - 2001226082

TP.HCM, tháng 06 năm 2025

BẢNG PHÂN CÔNG CÔNG VIỆC NHÓM 4

MSSV	Họ tên	Nội dung công việc
2001220330	Trần Duy Bảo	Word, Giao diện đăng nhập/đăng ký Chức năng quản lý sự kiện Chức năng thống kê.
2001226082	Nguyễn Võ Như Ý	API, Giao diện màn hình chính, hồ sơ cá nhân, tạo giao dịch, ví/tạo ví, quản lý nhóm, quản lý budget

Lời cảm ơn

Sau quá trình học tập và rèn luyện tại khoa Công nghệ thông tin trường Đại học Công Thương TP.HCM, em đã được trang bị các kiến thức cơ bản cùng các kỹ năng thực tế để có thể hoàn thành Đồ án của mình.

Để hoàn thành đồ án này, với lòng biết ơn sâu sắc em xin gửi lời cảm ơn chân thành đến:

Ban Giám hiệu trường Đại học Công Thương TP.HCM vì đã tạo điều kiện thuận lợi để sinh viên tìm kiếm, nghiên cứu thông tin với hệ thống thư viện hiện đại, đa dạng các loại sách và tài liệu.

Gần gũi hơn là những lời tốt đẹp nhất xin gửi đến thầy Đỗ Gia Bảo đã tận tình giúp đỡ, định hướng cách tư duy và hướng làm việc khoa học. Đó là những góp ý hết sức quý báu không chỉ trong quá trình thực hiện đồ án mà còn là hành trang tiếp bước cho em trong quá trình học tập và làm việc sau này.

Sau cùng, xin chúc quý Thầy Cô trong khoa Công nghệ Thông tin nói riêng cũng như các giáo viên tại trường Đại học Công Thương nói chung thật dồi dào sức khỏe, niềm tin để tiếp tục thực hiện sứ mệnh cao đẹp của mình.

This image shows a full page of white paper with horizontal dotted lines, typical of primary school writing paper. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

MỤC LỤC

Lời cảm ơn.....	3
Nhận xét của giảng viên	4
CHƯƠNG 1: GIỚI THIỆU TỔNG QUAN.....	8
1. Tổng quan đề tài:	8
1.1 Giới thiệu đề tài:.....	8
1.2 Phạm vi nghiên cứu:	8
1.3 Nội dung nghiên cứu:.....	8
1.4 Kết quả hướng tới:	8
2. Công cụ sử dụng:	9
CHƯƠNG 2: FLUTTER.....	10
1. Flutter là gì?	10
2. Các tính năng của Flutter.....	11
3. Kiến trúc của Flutter	12
3.1. Kiến trúc Flutter.....	12
3.2. Flutter Engine	12
3.3. Thư viện nền tảng (Foudation Library).....	13
3.4. Widget	13
4. Giới thiệu về ngôn ngữ lập trình Dart	14
4.1. Các từ khóa	14
4.2. Kiểu dữ liệu.....	14
4.3. Các biến và hàm	16
4.4. Toán tử (Operators).....	16
4.5. Câu lệnh điều kiện.....	19
4.6. Vòng lặp	20
4.7. Chú thích (Comment)	21
4.8. Continue và Break.....	22
4.9. Từ khóa Final và Const	23
4.10. Lập trình hướng đối tượng (OOP).....	24
5. Một số loại Widget thường gặp trong Flutter	31
5.1. Widget Flutter	31
5.2. Widget hiển thị	33

5.3.	Widget ẩn	36
6.	Tìm hiểu bố cục (layout) Trong giao diện Flutter.....	42
6.1.	Bố cục	42
6.2.	Bố cục một Widget	43
6.3.	Các loại Widget bố cục.....	44
7.	Tìm hiểu về Cử chỉ (Gestures) với giao diện Flutter	48
7.1.	Con trỏ.....	48
7.2.	Cử chỉ	49
7.3.	Dò cử chỉ.....	50
8.	Quản lý State	50
8.1.	Trạng thái tức thời	51
8.2.	Trạng thái ứng dụng	52
9.	Quản lý thư viện và các gói trong Flutter (Lib & Package).....	53
9.1.	Nạp thư viện trong Dart	53
9.2.	Các thư viện cung cấp sẵn	54
9.3.	Các gói hay sử dụng	55
10.	Tìm hiểu về Navigator và Routing trong Flutter	59
10.1.	Tạo routes	60
10.2.	Điều hướng sang route thứ hai bằng phương thức Navigator.push()	62
10.3.	Quay lại route đầu tiên bằng phương thức Navigator.pop()	62
10.4.	Điều hướng với các route được đặt tên.....	63
11.	Tìm hiểu về Database trong Flutter	66
11.1.	Cơ sở dữ liệu SQLite	67
11.2.	Firebase – NoSQL - API lưu trữ online.....	72
11.2.1.	Sơ lược về Firebase	72
11.2.2.	Giao tiếp API trong hệ thống.....	73
11.2.3.	Thêm Firebase vào ứng dụng Flutter của bạn	73
CHƯƠNG 3: XÂY DỰNG ỨNG DỤNG.....		75
1.	Tổng quan	75
1.1.	Tên ứng dụng.....	75
1.2.	Lý do chọn ứng dụng	75
1.3.	Đối tượng hướng đến	75
1.4.	Môi trường phát triển ứng dụng.....	75

1.5.	Kết quả mong đợi	76
1.6.	Khảo sát hiện trạng	76
1.7.	Quy trình thực hiện các công việc chính	77
2.	Phân tích, thiết kế hệ thống	78
2.1.	Xác định và mô hình hóa các yêu cầu phần mềm	78
2.2.	Thiết kế hệ thống	88
2.3.	Thiết kế giao diện	89
5.	Hướng dẫn sử dụng phần mềm	89
5.1.	Bắt đầu với hệ thống	89
5.2.	Đăng nhập, đăng ký và đăng xuất khỏi hệ thống	89
5.3.	Chuyển tiếp giữa các thành phần bên trong ứng dụng	99
5.4.	Thêm chi tiêu	99
5.5.	Sửa chi tiêu	99
CHƯƠNG 4. KẾT LUẬN		100
TÀI LIỆU THAM KHẢO		102

CHƯƠNG 1: GIỚI THIỆU TỔNG QUAN

1. Tổng quan đề tài:

1.1 Giới thiệu đề tài:

Với sự phát triển nhanh của smartphone như hiện nay với hai hệ điều hành phổ biến nhất là Android và IOS thì nhu cầu phát triển ứng dụng trên smartphone là rất lớn và việc phát triển riêng lẻ ứng dụng trên hai hệ điều hành trên bằng hai ngôn ngữ, hai framework khác nhau sẽ làm tốn rất nhiều chi phí và thời gian. Do đó việc có một framework cross-platform có thể code một lần và chạy trên cả hai hệ điều hành sẽ giúp tiết kiệm thời gian và tiền bạc hơn.

Hiện nay có rất nhiều framework cross-platform hỗ trợ phát triển ứng dụng trên cả hai hệ điều hành một cách nhanh chóng, tiện lợi nhưng vẫn đảm bảo tính bảo mật thông tin khách hàng. Và nổi bật trong những năm gần đây đó là Flutter.

1.2 Phạm vi nghiên cứu:

Trong đồ án lần này, em chủ yếu tập trung vào nghiên cứu tổng quan cách thức hoạt động và cách sử dụng các thư viện - package cũng như cách lưu trữ dữ liệu của Flutter. Đồng thời, em áp dụng các nghiên cứu trên vào một ứng dụng thực tế để giúp em có thể hiểu sâu hơn các kiến thức đã tìm hiểu.

1.3 Nội dung nghiên cứu:

Em sẽ tiến hành nghiên cứu chi tiết về cách thức hoạt động, ưu - khuyết điểm cũng như các thư viện - tính năng - package liên quan đến Flutter.

1.4 Kết quả hướng tới:

Với đề tài này, em đề ra hai mục tiêu chính:

- Đối với các nhân: mở rộng kiến thức của mình về Flutter thông qua quá trình tìm hiểu và áp dụng vào ứng dụng thực tế. Các kiến thức tìm hiểu được thông qua đồ án thúc đẩy em phát triển thêm các ứng dụng khác bằng Flutter. Đồng thời, em học được cách nghiên cứu và sử dụng một framework mới cần trải qua quá trình gì nhằm giúp em dễ dàng tiếp xúc với các công nghệ mới hơn trong tương lai.
- Đối với các lập trình viên dùng đồ án nghiên cứu này làm tài liệu tham khảo: thông qua tài liệu nghiên cứu và ứng dụng em đã xây dựng, các lập trình viên khác có thể dễ dàng định hướng cần phải tìm hiểu gì khi sử dụng Flutter. Đồng thời em có ghi một số khái niệm, kiến thức cơ

bản và thư viện phổ biến được đề xuất bởi cộng đồng khi sử dụng Flutter, các lập trình viên khác có thể tham khảo và tìm hiểu sâu hơn.

2. Công cụ sử dụng:

Trong quá trình xây dựng phần mềm, em đã sử dụng các phần mềm sau:

- **Android Studio:** Khởi tạo máy ảo
- **Visual Studio Code:** Code giao diện ứng dụng
- **Draw.io:** Vẽ các sơ đồ
- **Chrome:** Tìm hiểu thông tin

CHƯƠNG 2: FLUTTER

1. Flutter là gì?

Flutter là một bộ SDK đa nền tảng có thể hoạt động trên iOS và Android do Google phát triển được sử dụng để tạo ra các ứng dụng dành cho di động (native app).

Flutter gồm 2 thành phần quan trọng:

- Một SDK (Software Development Kit): Một bộ sưu tập các công cụ sẽ giúp bạn phát triển các ứng dụng của mình.
- Một Framework (UI Library based on widgets): Một tập hợp các thành phần giao diện người dùng (UI) có thể tái sử dụng (button, text inputs, slider, v.v.) giúp bạn có thể cá nhân hóa tùy theo nhu cầu của riêng mình.

Nói chung, tạo một ứng dụng di động là một công việc rất phức tạp và đầy thử thách. Có rất nhiều framework có sẵn, cung cấp các tính năng tuyệt vời để phát triển các ứng dụng di động. Để phát triển các ứng dụng dành cho thiết bị di động, Android cung cấp một framework gốc dựa trên ngôn ngữ Java và Kotlin, trong khi iOS cung cấp một framework dựa trên ngôn ngữ Objective-C/Swift.

Vì vậy, chúng ta cần hai ngôn ngữ và framework khác nhau để phát triển ứng dụng cho cả hai hệ điều hành. Ngày nay, để khắc phục sự phức tạp này, có một số framework đã được giới thiệu hỗ trợ cả hệ điều hành cùng với các ứng dụng dành cho máy tính để bàn. Những loại framework này được gọi là công cụ phát triển đa nền tảng.

Framework phát triển đa nền tảng có khả năng viết một code và có thể triển khai trên nhiều nền tảng khác nhau (Android, iOS và Máy tính để bàn). Nó tiết kiệm rất nhiều thời gian và nỗ lực phát triển của các nhà phát triển.

Có một số công cụ có sẵn để phát triển đa nền tảng, bao gồm các công cụ dựa trên web. Mỗi framework này có mức độ thành công khác nhau trong ngành công nghiệp di động. Gần đây, một framework công tác mới đã được giới thiệu trong họ phát triển đa nền tảng có tên là Flutter được phát triển từ Google.

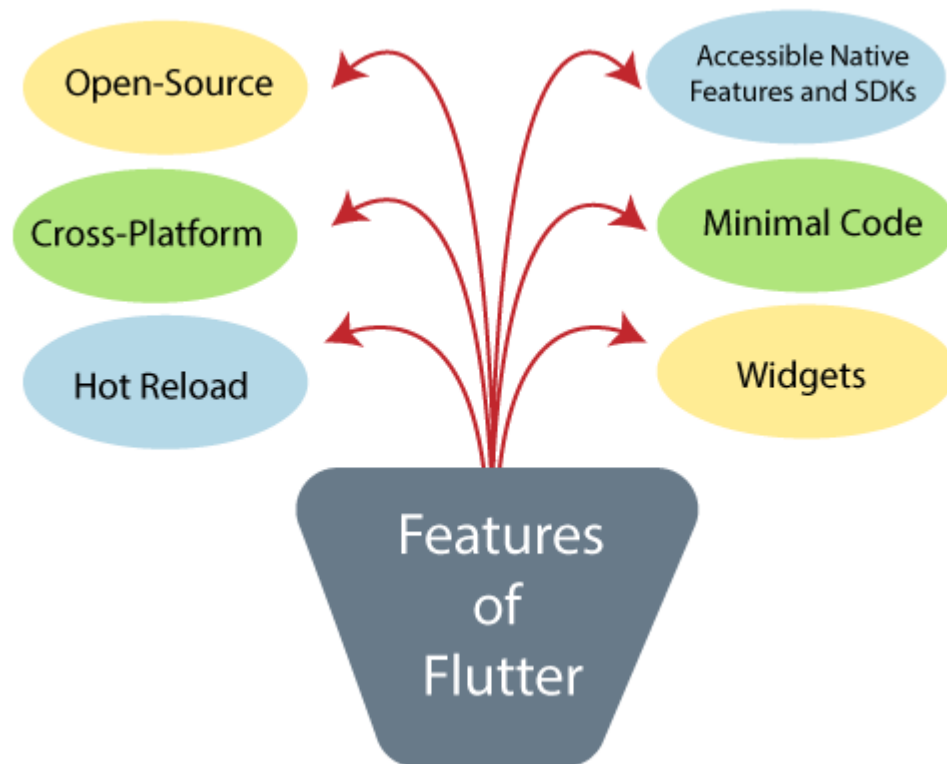
Flutter là một bộ công cụ giao diện người dùng để tạo các ứng dụng nhanh, đẹp, được biên dịch nguyên bản cho thiết bị di động, web và máy tính để bàn với một ngôn ngữ lập trình và cơ sở code duy nhất. Nó là miễn phí và code nguồn mở.

Ban đầu nó được phát triển từ Google và bây giờ được quản lý theo tiêu chuẩn ECMA. Ứng dụng Flutter sử dụng ngôn ngữ lập trình Dart để tạo ứng dụng.

Flutter chủ yếu được tối ưu hóa cho các ứng dụng di động 2D có thể chạy trên cả nền tảng Android và iOS. Chúng ta cũng có thể sử dụng nó để xây dựng các ứng dụng đầy đủ tính năng, bao gồm máy ảnh, bộ nhớ, vị trí địa lý, mạng, SDK của bên thứ ba, v.v...

2. Các tính năng của Flutter

Flutter cung cấp các phương pháp dễ dàng và đơn giản để bắt đầu xây dựng các ứng dụng dành cho thiết bị di động và máy tính để bàn đẹp mắt với một bộ thiết kế material design và widget phong phú. Ở đây, chúng ta sẽ thảo luận về các tính năng chính của nó để phát triển framework di động.



Hình 1.1: Các tính năng của Flutter

Code nguồn mở (Open-Source): Flutter là một framework code nguồn mở và miễn phí để phát triển các ứng dụng di động.

Đa nền tảng (Cross-platform): Tính năng này cho phép Flutter viết code một lần, duy trì và có thể chạy trên các nền tảng khác nhau. Nó tiết kiệm thời gian, công sức và tiền bạc của các nhà phát triển.

Tải lại nóng (Hot Reload): Bất cứ khi nào nhà phát triển thực hiện thay đổi trong code, thì những thay đổi này có thể được nhìn thấy ngay lập tức với Tải lại nóng. Nó có nghĩa là những thay đổi hiển thị ngay lập tức trong chính ứng dụng. Đây là một tính năng rất tiện dụng, cho phép nhà phát triển sửa các lỗi ngay lập tức.

Các tính năng và SDK gốc có thể truy cập (Accessible Native Features and SDKs): Tính năng này cho phép quá trình phát triển ứng dụng dễ dàng và thú vị thông qua code gốc của Flutter, tích hợp bên thứ ba và các API nền tảng. Do đó, chúng tôi có thể dễ dàng truy cập SDK trên cả hai nền tảng.

Code tối thiểu (Minimal code): Ứng dụng Flutter được phát triển bởi ngôn ngữ lập trình Dart, sử dụng biên dịch JIT và AOT để cải thiện thời gian khởi động tổng thể, hoạt động và tăng tốc hiệu suất. JIT nâng cao hệ thống phát triển và làm mới giao diện người dùng mà không cần nỗ lực thêm vào việc xây dựng hệ thống mới.

Widget: framework công tác Flutter cung cấp các widget có khả năng phát triển các thiết kế cụ thể có thể tùy chỉnh. Quan trọng nhất, Flutter có hai bộ widget: Material Design và các widget Cupertino giúp mang lại trải nghiệm không có trục trặc trên tất cả các nền tảng.

3. Kiến trúc của Flutter

3.1. Kiến trúc Flutter

Trong phần này, chúng ta sẽ thảo luận về kiến trúc của Flutter framework. Kiến trúc Flutter chủ yếu bao gồm bốn thành phần.

- Động cơ Flutter (Flutter Engine)
- Thư viện nền tảng (Foundation Library)
- Vật dụng (Widgets)
- Thiết kế các widget cụ thể (Design Specific Widgets)

3.2. Flutter Engine

Nó là một công cụ để giúp chạy các ứng dụng di động chất lượng cao và cơ bản dựa trên ngôn ngữ C ++. Nó triển khai các thư viện lõi Flutter bao gồm animation và đồ họa, tệp và mạng I / O, kiến trúc plugin, hỗ trợ trợ năng và thời gian chạy dart để phát triển, biên dịch và chạy các ứng dụng Flutter. Phải sử

dùng thư viện đồ họa mã nguồn mở của Google, Skia, để hiển thị đồ họa cấp thấp.

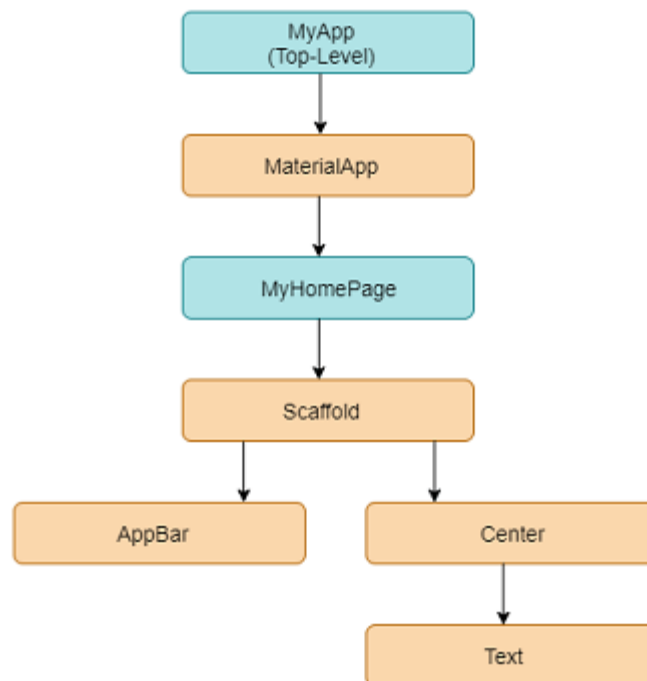
3.3. Thư viện nền tảng (Foudation Library)

Nó chứa tất cả các gói cần thiết cho các khối build cơ bản để viết một ứng dụng Flutter. Các thư viện này được viết bằng ngôn ngữ Dart.

3.4. Widget

Trong Flutter, mọi thứ đều là một widget, đó là khái niệm cốt lõi của framework. Widget trong Flutter về cơ bản là một thành phần giao diện người dùng ảnh hưởng và kiểm soát chế độ xem và giao diện của ứng dụng. Nó đại diện cho một mô tả bất biến về một phần của giao diện người dùng và bao gồm đồ họa, văn bản, hình dạng và animation được tạo bằng các widget. Các widget tương tự như các thành phần React.

Trong Flutter, ứng dụng tự nó là một widget chứa nhiều widget con. Điều đó có nghĩa rằng ứng dụng là tiện ích con cấp cao nhất và giao diện người dùng của nó được xây dựng bằng cách sử dụng một hoặc nhiều tiện ích con, bao gồm các tiện ích con phụ. Tính năng này giúp bạn tạo một giao diện người dùng phức tạp rất dễ dàng.



Hình 1.2: Kiến trúc Flutter

Trong ví dụ trên, chúng ta có thể thấy rằng tất cả các thành phần đều là các widget có chứa các widget con. Do đó, ứng dụng Flutter tự nó là một widget.

4. Giới thiệu về ngôn ngữ lập trình Dart**4.1. Các từ khóa**

abstract	else	import	show
as	enum	in	static
assert	export	interface	super
async	extends	is	switch
await	extension	late	sync
break	external	library	this
case	factory	mixin	throw
catch	false	new	true
class	final	null	try
const	finally	on	typedef
continue	for	operator	var
covariant	Function	part	void
default	get	required	while
deferred	hide	rethrow	with
do	if	return	yield
dynamic	implements	set	

4.2. Kiểu dữ liệu

Dart là một ngôn ngữ lập trình Strongly Typed. Nó có nghĩa là, mỗi giá trị bạn sử dụng trong ngôn ngữ lập trình của mình có một kiểu là chuỗi hoặc số và phải được biết chính xác là kiểu dữ liệu gì trước khi code được biên dịch. Ở đây, chúng ta sẽ thảo luận về các kiểu dữ liệu cơ bản phổ biến nhất được sử dụng trong ngôn ngữ lập trình Dart.

Loại dữ liệu	Ví dụ	Mô tả
String	String myName = 'duybaoTDB';	Được dùng để biểu diễn chuỗi ký tự. Nó được nhập vào trong cặp nháy đơn ' ' hoặc nháy kép

		" ". Dùng ký hiệu \, \' để biểu diễn ký tự nhảy đơn (') trong. Khi bạn quyết định dấu ngoặc kép, bạn phải nhất quán với lựa chọn của mình. Nếu bạn muốn chuỗi của mình bao gồm nhiều dòng thì sử dụng "" để chứa chuỗi bên trong.
int	int age = 21;	Biểu diễn dạng số nguyên với từ khóa int. Kiểu int được biểu diễn tối đa 64-bit phụ thuộc vào phần cứng.
double	double height = 1.8;	Biểu diễn dạng số thực dấu phẩy động với từ khóa double. double đều được biểu diễn bởi 64bit.
bool	bool check = true;	Sử dụng từ khóa bool để biểu thị giá trị Boolean true và false.
List	List<int> num = [1, 2, 3, 4];	List<T> biểu diễn một nhóm các đối tượng được sắp xếp và có thiết kế giống như mảng (array) trong các ngôn ngữ khác.
Map	Map<String, dynamic> person = {"name": "Bảo", "age": 18, "height": 1.8};	Map<T, T> kiểu dữ liệu biểu diễn một tập đối tượng không được sắp xếp ở dạng <key, value>.
dynamic	dynamic name = "duybao"; dynamic age = 21;	dynamic trong Dart có thể hiểu là kiểu dữ liệu tùy chọn. Đây là kiểu cơ bản cho mọi kiểu dữ liệu trong Dart. Kiểu dữ liệu của một biến nếu không được chỉ định rõ ràng thì sẽ được gán với từ khóa dynamic.

class	Person person = Person();	Nói chung, mọi thứ trong Dart là một đối tượng (ví dụ: Số nguyên, Chuỗi). Nhưng một đối tượng cũng có thể phức tạp hơn.
-------	---------------------------	---

Bảng 1.1: Các kiểu dữ liệu trong ngôn ngữ Dart

4.3. Các biến và hàm

Các biến là không gian tên trong bộ nhớ lưu trữ các giá trị. Tên của một biến được gọi là định danh (identifiers). Chúng là nơi chứa dữ liệu, có thể lưu trữ giá trị của bất kỳ kiểu nào.

Ví dụ: Khi khai báo một biến `var myAge = 50;`

Ở đây, `myAge` là một biến lưu trữ giá trị số nguyên 50. Chúng ta cũng có thể cho nó là `int` và `double`. Tuy nhiên, Dart có một tính năng Type Inference, suy luận các loại giá trị. Vì vậy, nếu bạn tạo một biến với từ khóa `var`, Dart có thể suy ra biến đó thuộc loại số nguyên.

Bên cạnh biến, Hàm là một tính năng cốt lõi khác của bất kỳ ngôn ngữ lập trình nào. Hàm là một tập hợp các câu lệnh thực hiện một nhiệm vụ cụ thể. Chúng được tổ chức thành các khối mã logic có thể đọc được, có thể bảo trì và có thể tái sử dụng. Khai báo hàm chứa tên hàm, kiểu trả về và các tham số. Ví dụ sau giải thích hàm được sử dụng trong lập trình Dart.

4.4. Toán tử (Operators)

Ngôn ngữ Dart hỗ trợ tất cả các toán tử, toán tử là để xác định cách thức làm việc giữa các toán hạng. Toán hạng có thể là một hằng số, biến số hoặc một lời gọi hàm... Tên của Operators được liệt kê dưới đây:

- Số học
- Tăng và giảm
- Logic
- So sánh

4.4.1. Toán tử số học

Toán tử số học là những phép toán cộng, trừ, nhân, chia cơ bản như trong toán học.

Giả sử biến `A` giữ giá trị 10, biến `B` giữ giá trị 20 thì:

Toán tử	Miêu tả	Ví dụ
+	Cộng hai toán hạng	$A + B$, kết quả là 30
-	Trừ toán hạng đầu cho toán hạng thứ hai	$A - B$, kết quả là -10
*	Nhân hai toán hạng	$A * B$, kết quả là 200
/	Chia toán hạng đầu cho toán hạng thứ hai	A / B , kết quả là 0.5
~/	Phép chia lấy phần nguyên	$B ~/ A$, kết quả là 2
%	Phép lấy số dư	$B \% A$, kết quả là 0

Bảng 1.2: Các toán tử toán học trong Dart**4.4.2. Toán tử tăng và giảm**

Một toán tử tăng hay toán tử giảm được sử dụng như là một phần của biểu thức, thì sẽ có một sự khác nhau quan trọng giữa dạng tiền tố và hậu tố. Nếu bạn sử dụng dạng tiền tố thì toán tử tăng hoặc toán tử giảm được thực hiện trước biểu thức, và nếu bạn sử dụng dạng hậu tố thì toán tử tăng hoặc toán tử giảm được thực hiện sau khi biểu thức được ước lượng.

Giả sử biến A giữ giá trị 10, biến B giữ giá trị 20 thì:

Toán tử	Miêu tả	Ví dụ
++	Toán tử tăng (++), tăng giá trị toán hạng thêm một đơn vị	$A++$, kết quả là 11 $++A$, kết quả là 11
--	Toán tử giảm (--), giảm giá trị toán hạng đi một đơn vị	$B--$, kết quả là 19 $--B$, kết quả là 19

Bảng 1.3: Toán tử tăng và toán tử giảm trong Dart**4.4.3. Toán tử logic**

Toán tử Logic được sử dụng để kiểm tra tính đúng đắn của một hoặc nhiều biểu thức. Giá trị trả về của các biểu thức này là một giá trị kiểu boolean, true hoặc false.

Toán tử	Miêu tả	Ví dụ
&&	Được gọi là toán tử logic AND (và). Kết quả là true nếu cả hai toán tử là true.	$(true \&\& true)$ là true $(true \&\& false)$ là false
	Được gọi là toán tử logic OR (hoặc). Kết quả là true nếu một trong hai toán tử là true.	$(true false)$ là true

		(false false) là false
!	Được gọi là toán tử NOT (phủ định). Sử dụng để đảo ngược lại trạng thái logic của toán hạng đó. Nếu điều kiện toán hạng là true thì phủ định nó sẽ là false.	!(true) là false !(false) là true

Bảng 1.4: Toán tử logic trong Dart

4.4.4. Toán tử so sánh

Toán tử so sánh dùng để so sánh hai toán hạng (2 biến giá trị) với nhau. Kết quả trả lại của toán tử so sánh nếu đúng thì sẽ là 1 (True), còn nếu sai thì kết quả sẽ là 0 (False).

Toán tử	Miêu tả	Ví dụ
==	So sánh hai toán hạng nếu bằng nhau kết quả sẽ là true và ngược lại sẽ là false	(5 == 5) kết quả là true (5 == 6) kết quả là false
!=	So sánh hai toán hạng nếu bằng nhau kết quả sẽ là false và ngược lại sẽ là true	(5 != 5) kết quả là false (5 != 6) kết quả là true
>	So sánh hai toán hạng nếu toán hạng thứ nhất lớn hơn toán hạng thứ hai thì sẽ là true và ngược lại sẽ là false	(5 > 6) kết quả là false (6 > 5) kết quả là true
<	So sánh hai toán hạng nếu toán hạng thứ nhất bé hơn toán hạng thứ hai thì sẽ là true và ngược lại sẽ là false	(5 < 6) kết quả là true (6 < 5) kết quả là false
>=	So sánh hai toán hạng nếu toán hạng thứ nhất lớn hơn hoặc bằng toán hạng thứ hai thì sẽ là true và ngược lại sẽ là false	(5 >= 6) kết quả là false (6 >= 5) kết quả là true (5 >= 5) kết quả là true

$\leq$	So sánh hai toán hạng nếu toán hạng thứ nhất bé hơn hoặc bằng toán hạng thứ hai thì sẽ là true và ngược lại sẽ là false	(5 $\leq$ 6) kết quả là true (6 $\leq$ 5) kết quả là false (5 $\leq$ 5) kết quả là true
--------	---	---

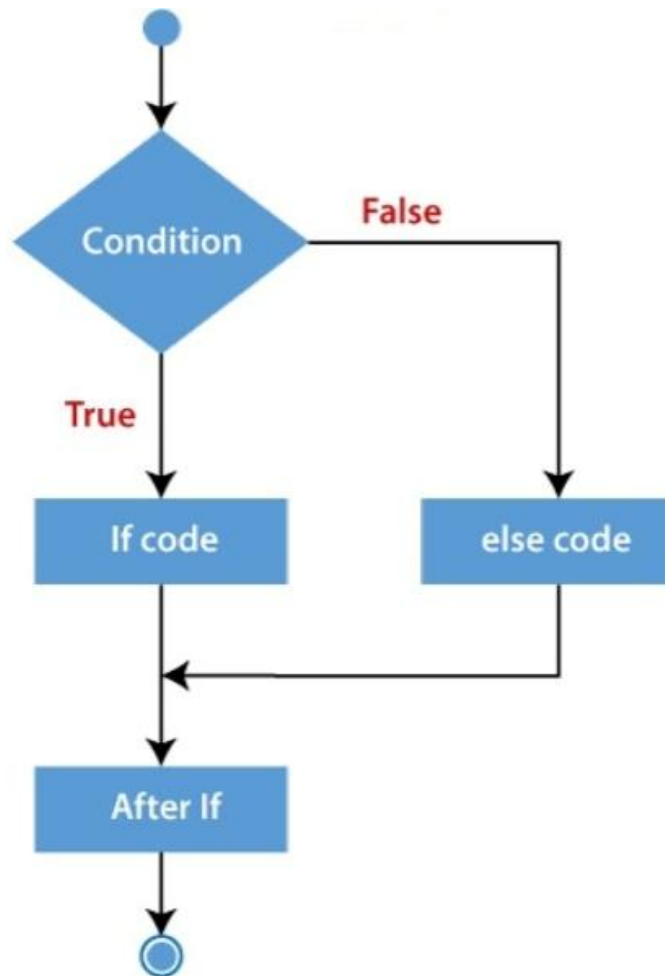
Bảng 1.5: Toán tử so sánh trong Dart

4.5. Câu lệnh điều kiện

Ngôn ngữ Dart hỗ trợ các loại câu lệnh ra quyết định là câu lệnh if, câu lệnh if-else. Đoạn code minh họa các câu lệnh ra quyết định if, if-else.



```
1 void main() {  
2   var num = 10;  
3   if (num % 2 == 0) {  
4     print("$num là số chẵn");  
5   } else {  
6     print("$num là số lẻ");  
7   }  
8 }
```



Hình 1.3: Sơ đồ câu lệnh điều kiện

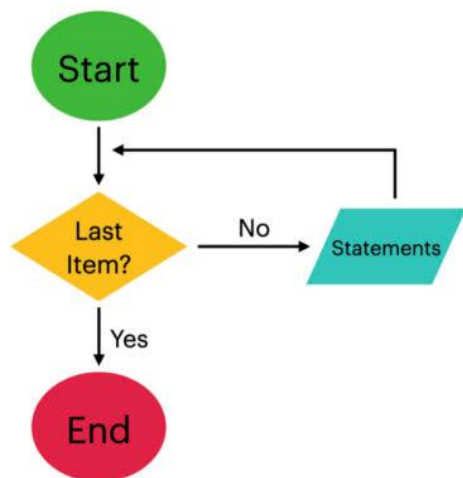
4.6. Vòng lặp

Các vòng lặp được sử dụng để thực thi một khối code lặp đi lặp lại cho đến khi một điều kiện được chỉ định trở thành đúng. Ngôn ngữ Dart hỗ trợ các loại câu lệnh lặp for, for..in, while, do..while.

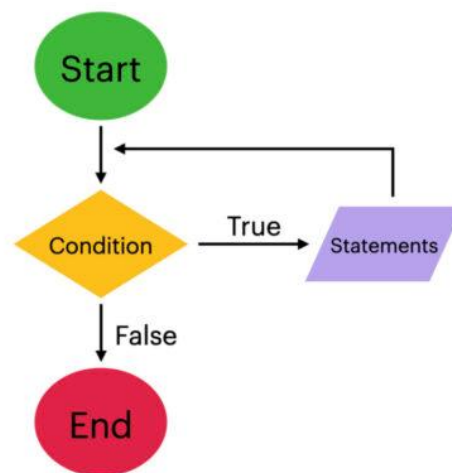


```
1 void main() {  
2     var list = ["dog", "cat", "fish", "duck"];  
3  
4     for (var element in list) {  
5         print(element);  
6     }  
7 }
```

For Loop



While Loop

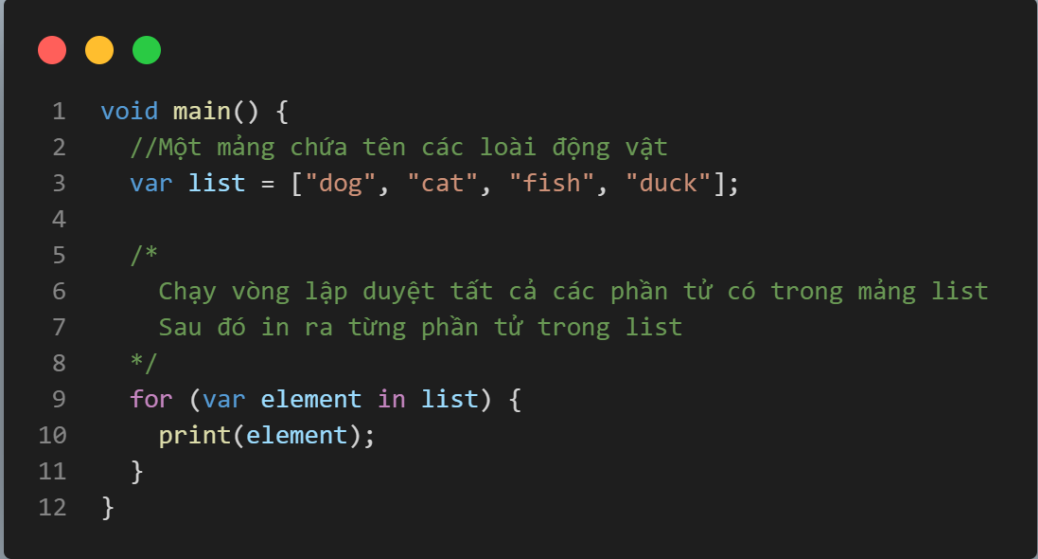


Hình 1.4: Sơ đồ vòng lặp

4.7. Chú thích (Comment)

Việc sử dụng chú thích là để cung cấp thông tin về dự án, biến hoặc một hoạt động, có ba bình luận được dùng ở trong lập trình Dart:

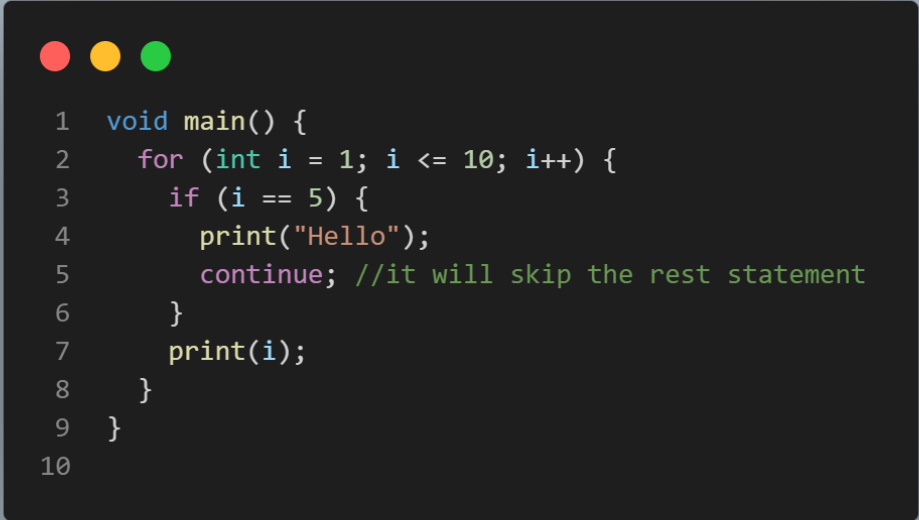
- **Thực hiện các chú thích định dạng:** Đó là một nhận xét dòng đơn (//)
- **Khối chú thích:** Đó là một nhận xét nhiều dòng (/*...*/)
- **Chú thích Tài liệu:** Đây là nhận xét tài liệu được sử dụng để mô tả cho thành viên và các kiểu (///)



```
1 void main() {
2     //Một mảng chứa tên các loài động vật
3     var list = ["dog", "cat", "fish", "duck"];
4
5     /*
6     Chạy vòng lặp duyệt tất cả các phần tử có trong mảng list
7     Sau đó in ra từng phần tử trong list
8     */
9     for (var element in list) {
10         print(element);
11     }
12 }
```

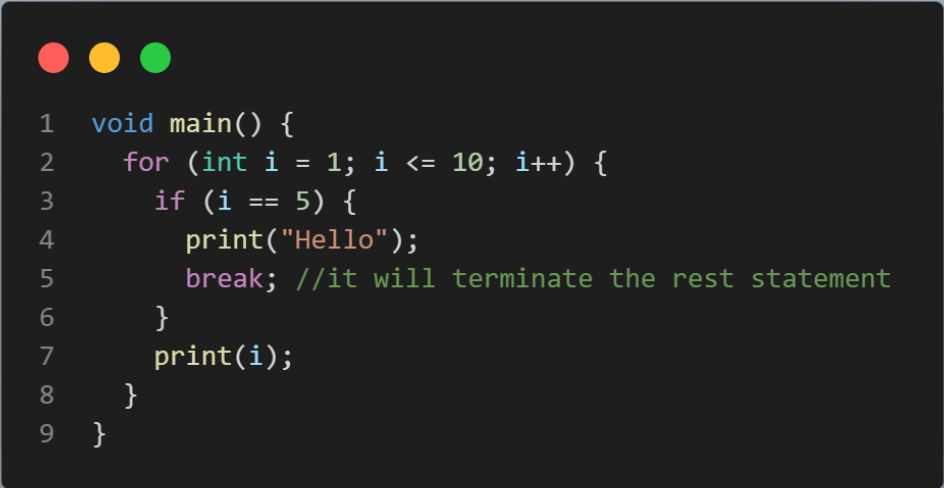
4.8. Continue và Break

Dart cũng đã sử dụng từ khóa Continue và Break trong vòng lặp, và ở những nơi khác nó được yêu cầu. Câu lệnh continue cho phép bạn bỏ qua đoạn code còn lại bên trong vòng lặp và ngay lập tức chuyển đến bước lặp tiếp theo của vòng lặp.



```
1 void main() {
2     for (int i = 1; i <= 10; i++) {
3         if (i == 5) {
4             print("Hello");
5             continue; //it will skip the rest statement
6         }
7         print(i);
8     }
9 }
10
```

Câu lệnh break cho phép bạn kết thúc hoặc dừng dòng hiện tại của một chương trình và tiếp tục thực hiện sau phần thân của vòng lặp.



```
1 void main() {
2     for (int i = 1; i <= 10; i++) {
3         if (i == 5) {
4             print("Hello");
5             break; //it will terminate the rest statement
6         }
7         print(i);
8     }
9 }
```

4.9. Từ khóa Final và Const

Chúng ta có thể sử dụng một từ khóa Final để hạn chế người dùng. Nó có thể được áp dụng trong nhiều ngữ cảnh, chẳng hạn như biến, lớp và phương thức.

Từ khóa Const dùng để khai báo hằng. Chúng ta không thể thay đổi giá trị của từ khóa const sau khi gán nó.



```
1 void main() {  
2   final a = 100;  
3   const pi = 3.14;  
4   print(a);  
5   print(pi);  
6 }
```

4.10. Lập trình hướng đối tượng (OOP)

Dart là một ngôn ngữ lập trình hướng đối tượng, có nghĩa là mọi giá trị trong Dart đều là một đối tượng. Một số cũng là một đối tượng trong ngôn ngữ Dart. Lập trình Dart hỗ trợ khái niệm về các tính năng OOP như đối tượng, lớp, giao diện, v.v

4.10.1. Đối tượng là gì trong lập trình

Là các đối tượng trong thế giới thực mà có thể mô tả thông qua thuộc tính và hành vi riêng của nó.

Ví dụ:

- Con người (thuộc tính: tên , tuổi , giới tính..., hành vi: chạy, đá, đấm, làm việc...)
- Căn nhà (thuộc tính: vị trí, diện tích, tên chủ nhà.. , hành vi: tránh nắng, giữ ấm...)
- Trong lập trình, khi bạn muốn xây dựng một đối tượng, trước tiên bạn cần phải xác định được: thuộc tính và các hành vi của nó.

4.10.2. Lớp (class) trong ngôn ngữ Dart

Class là dùng để chứa các biến, và các hàm. Trong biểu diễn một đối tượng, class dùng để mô tả đối tượng, bao gồm các biến (các thuộc tính của đối tượng) các trường dữ liệu và các hàm (các phương thức của đối tượng).

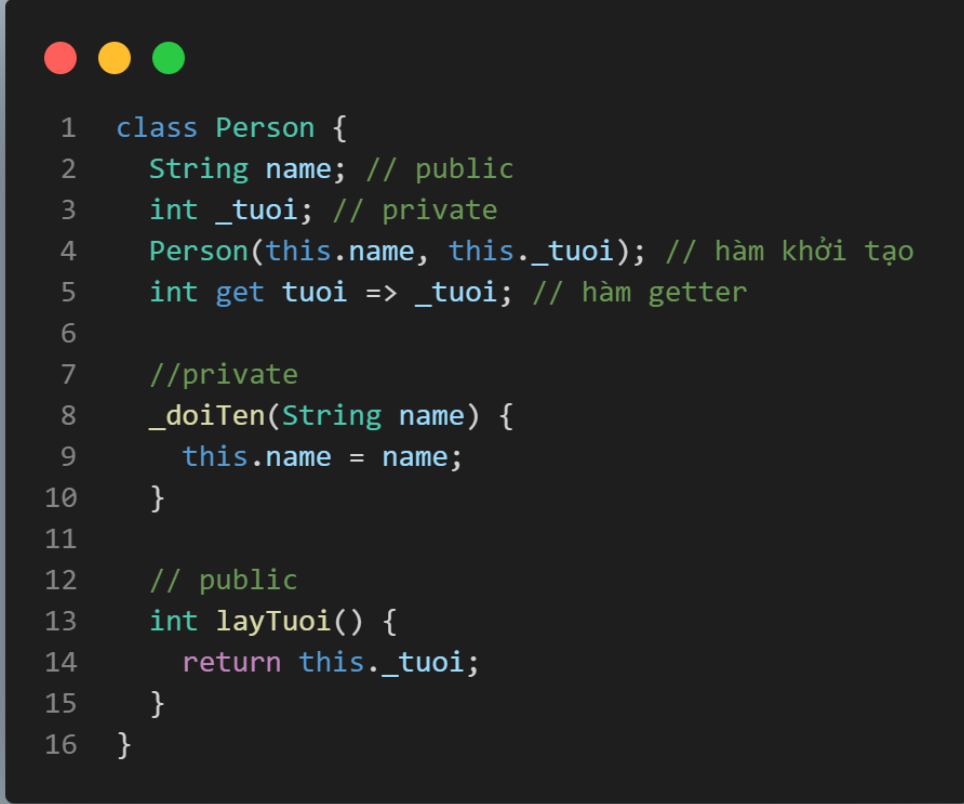
Điểm khác biệt của ngôn ngữ Dart:

- Không giống như các ngôn ngữ khác như Java hay C#, Dart cho phép khai báo nhiều đối tượng trong một file Dart.
- Không dùng các từ khoá như : public, private, protected.
- Chỉ cho phép tạo một constructor.

Private được biểu diễn bằng dấu “_” trước các biến hay hàm. Ví dụ: String _name; nó được hiểu như : private String name;

Trong class có các thành phần:

Thuộc tính: Thuộc tính là những thông tin riêng của mỗi đối tượng, ta có thể thấy nó như là những biến liên quan đến đối tượng đó.



```
1 class Person {
2   String name; // public
3   int _tuoi; // private
4   Person(this.name, this._tuoi); // hàm khởi tạo
5   int get tuoi => _tuoi; // hàm getter
6
7   //private
8   _doiTen(String name) {
9     this.name = name;
10  }
11
12  // public
13  int layTuoi() {
14    return this._tuoi;
15  }
16 }
```

Cách dùng “=>” trong ngôn ngữ Dart, chúng ta thấy có hàm getter:

```
int get tuoi => _tuoi;
```

Cách viết trên tương tự với:

```
int get tuoi {  
    return _tuoi;  
}
```

Phương thức khởi tạo: là những phương thức đặc biệt được gọi đến ngay khi khởi tạo 1 đối tượng nào đó. Đặc điểm của phương thức khởi tạo là : Có tên trùng với tên lớp, không có kiểu trả về, được tự động gọi khi 1 đối tượng thuộc lớp được khởi tạo, và có thể có nhiều Constructor. Cũng giống như các ngôn ngữ khác, Dart cũng có hàm khởi tạo. Nếu developer không tạo hàm khởi tạo riêng thì hàm khởi tạo mặc định không biến đầu vào sẽ được sử dụng.

Cách thức để viết một hàm khởi tạo trong Dart.

```
Person (this.name, this._tuoi);
```

Thay cho cách viết hàm khởi tạo của các ngôn ngữ khác:

```
Person (String name, int tuoi){  
    this.name = name;  
    this._tuoi = tuoi;  
}
```

Ví dụ định nghĩa một lớp trong Dart:

```
1  class Mobile {
2      // Property Declaration
3      String color, brandName, modelName;
4
5      Mobile(this.color, this.brandName, this.modelName);
6
7      // Method Creation
8      String calling() {
9          return "Mobile can do call to everyone.";
10     }
11
12     String musicPlay() {
13         return "Mobile can play all types of Music.";
14     }
15
16     String clickPicture() {
17         return "Mobile can take pictures.";
18     }
19 }
20
21 void main() {
22     // Object Creation
23     var myMob = new Mobile("black", "SamSung", "S22 Ultra");
24
25     // Accessing Class's Property
26     myMob.color = "Black";
27     myMob.brandName = "Apple Inc.";
28     myMob.modelName = "iPhone 11 Pro";
29
30     //Display Output
31     print(myMob.color);
32     print(myMob.modelName);
33     print(myMob.brandName);
34     print(myMob.calling());
35     print(myMob.musicPlay());
36     print(myMob.clickPicture());
37 }
```

Trong ví dụ trên, chúng ta định nghĩa một lớp Mobile, có ba biến kiểu chuỗi và ba hàm hoặc phương thức. Sau đó, chúng ta tạo một hàm main mà Dart sẽ thực thi đầu tiên khi ứng dụng của bạn khởi động. Bên trong main, chúng ta tạo một đối tượng để truy cập các thuộc tính của lớp. Cuối cùng, chúng ta in đầu ra.

4.10.3. Các đặc tính trong lập trình hướng đối tượng Dart

Tính trừu tượng (Abstraction)

Tính trừu tượng thể hiện ở việc lựa chọn các thuộc tính và hành vi của đối tượng mà không phải liệt kê hết tất cả các thuộc tính và hành vi của đối tượng đó.

Ví dụ: Để mô tả một người có rất nhiều thuộc tính và hành vi. Nhưng chúng ta chỉ sử dụng các thuộc tính như: tên, năm sinh, quê quán và thuộc tính như: đi, chạy mà không cần liệt kê hết tất cả các thuộc tính và hành vi khác như: tình trạng hôn nhân, lái xe, đá, đấm...

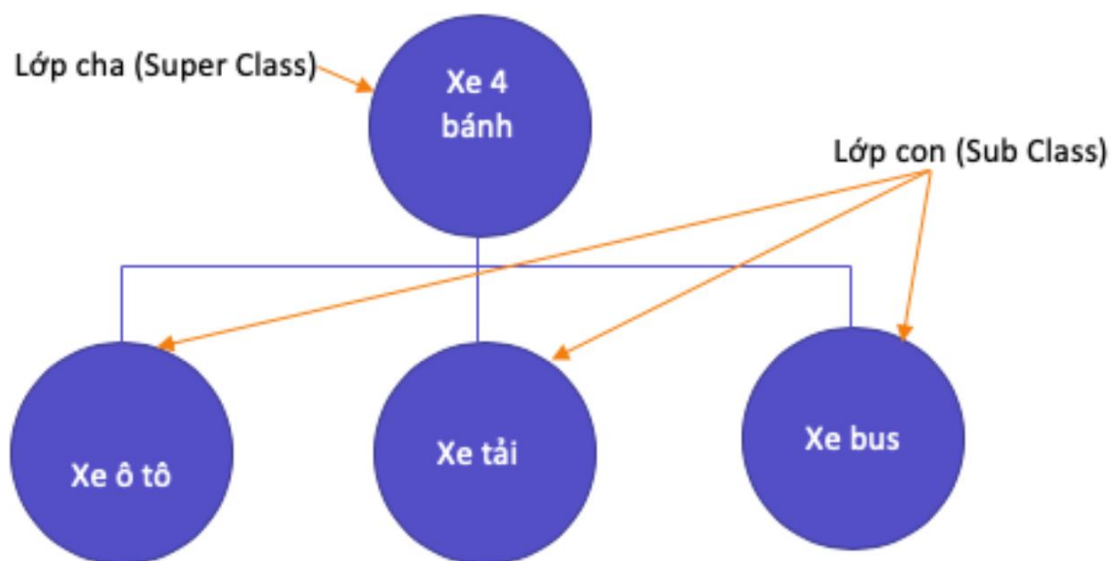
Tính đóng gói (Encapsulation)

Tính đóng gói thể hiện sự che giấu trong đối tượng với mục đích bảo vệ dữ liệu và tăng khả năng mở rộng. Vì vậy khi triển khai một đối tượng, các thuộc tính nên dùng tính năng private. Ví dụ: `int _tuoi;` và sử dụng các phương thức để truy xuất các trường dữ liệu như getter, setter:

```
1  int _tuoi;  
2  
3  // getter  
4  int get tuoi => _tuoi;  
5  
6  // setter  
7  void set datTuoi(int tuoi)  
8  {  
9      this._tuoi = tuoi;  
10 }
```

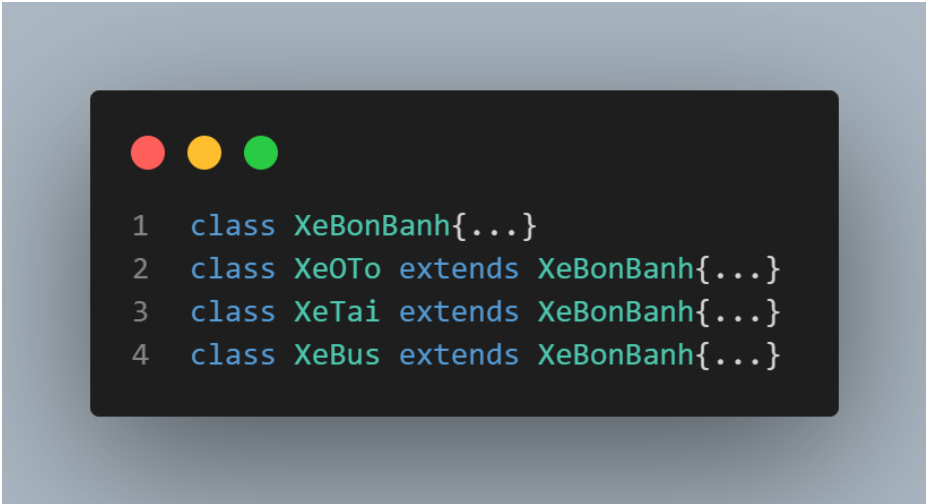
Tính kế thừa (Inheritance)

Trong một phần mềm hay chương trình, được cấu tạo bởi nhiều lớp khác nhau cùng các thành phần khác. Mỗi quan hệ giữa các lớp, có mối quan hệ kế thừa, gồm lớp cha (super class) và các lớp con (sub class), Các lớp con đó lại có thể là lớp cha của các lớp khác.



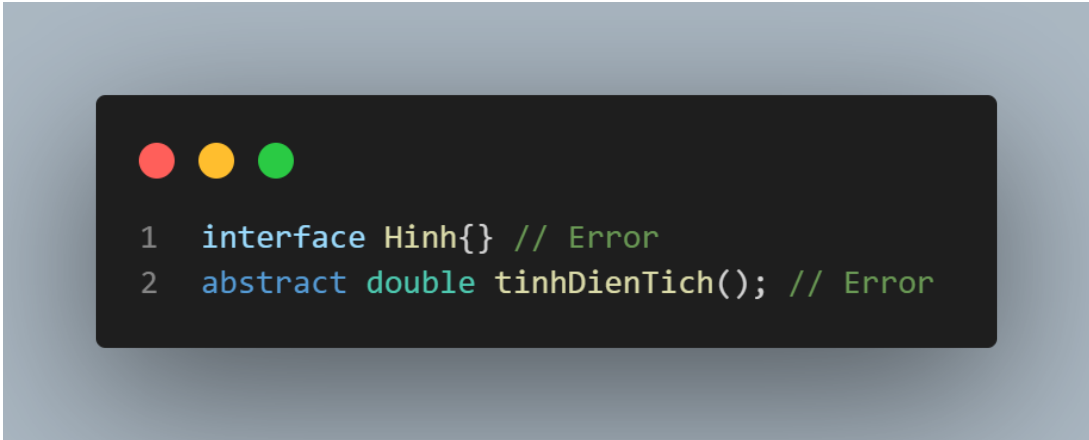
Hình 1.5: Mô tả tính kế thừa giữa các lớp

Mục đích của kế thừa là tái sử dụng. Lớp con có thể sở hữu các thuộc tính và phương thức public của lớp cha nhưng không được sở hữu các thuộc tính hay phương thức private và các hàm constructor. Biểu diễn kế thừa trong Dart cũng tương tự như trong Java : dùng extends. Cách kế thừa như sau:



```
1 class XeBonBanh{...}
2 class XeOTo extends XeBonBanh{...}
3 class XeTai extends XeBonBanh{...}
4 class XeBus extends XeBonBanh{...}
```

Trong ngôn ngữ Dart, không dùng từ khoá interface hay không dùng phương thức có từ khoá abstract ở phía trước.



```
1 interface Hình{} // Error
2 abstract double tinhDienTich(); // Error
```

Tính đa hình (Polymorphism)

Tính đa hình trong ngôn ngữ Dart cũng có ý nghĩa giống như trong các ngôn ngữ khác. Cùng biểu diễn một hành vi nhưng từng lớp có cách biểu diễn khác nhau. Để hiểu rõ hơn về tính đa hình của lập trình hướng đối tượng thì hãy xem một đoạn code minh họa về tính năng của xe.

```
1 // Khai báo về tính năng của Xe
2 abstract class Xe {
3     void chuyenCho(); // Chuyên chở
4 }
5
6 // Khai báo của Xe tải
7 class XeTai extends Xe {
8     @override
9     void chuyenCho() {
10         print(" Chỉ chở hàng");
11     }
12 }
13
14 // Khai báo về Xe khách
15 class XeKhach extends Xe {
16     @override
17     void chuyenCho() {
18         print(" Chỉ chở người");
19     }
20 }
```

Như vậy, Cùng một hành vi là Chở, nhưng với các loại xe là các công dụng lại khác nhau – Đó là tính đa hình.

5. Một số loại Widget thường gặp trong Flutter

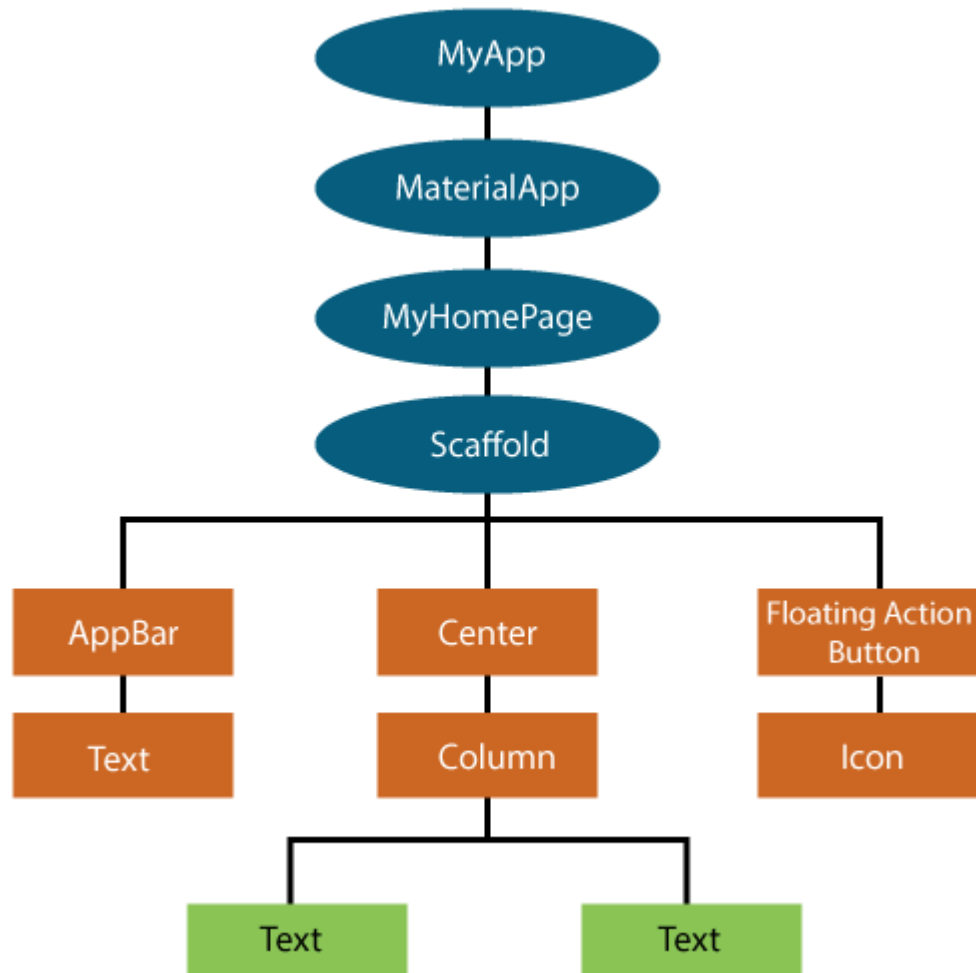
5.1. Widget Flutter

Trong phần này, chúng ta sẽ tìm hiểu khái niệm về một widget, cách tạo nó và các loại khác nhau của chúng có sẵn trong Flutter framework. Chúng ta đã biết trước đó rằng mọi thứ trong Flutter đều là một widget.

Bất cứ khi nào bạn định viết mã để xây dựng bất cứ thứ gì trong Flutter, nó sẽ nằm trong một widget. Mục đích chính là xây dựng ứng dụng từ các widget. Nó mô tả chế độ xem ứng dụng của bạn trông như thế nào với cấu hình và trạng thái hiện tại của chúng.

Khi bạn thực hiện bất kỳ thay đổi nào trong code, widget con sẽ xây dựng lại mô tả của nó bằng cách tính toán sự khác biệt của widget con hiện tại và trước đó để xác định những thay đổi tối thiểu đối với việc hiển thị trong giao diện người dùng của ứng dụng.

Các widget được lồng vào nhau để xây dựng ứng dụng. Nó có nghĩa là thư mục gốc của ứng dụng của bạn tự nó là một widget, và tất cả các cách nhìn xuống cũng là một widget. Ví dụ: một widget có thể hiển thị một thứ gì đó, có thể xác định thiết kế, có thể xử lý tương tác, v.v.



Hình 1.6: Cây Widget

5.2. Widget hiển thị

Các widget hiển thị có liên quan đến dữ liệu đầu vào và đầu ra của người dùng. Một số loại quan trọng của widget con này là:

Text

Text (Văn bản) là một widget con trong Flutter cho phép chúng ta hiển thị một chuỗi Text với một dòng duy nhất trong ứng dụng của chúng ta. Tùy thuộc vào các ràng buộc về bố cục, chúng ta có thể ngắt chuỗi trên nhiều dòng hoặc tất cả có thể được hiển thị trên cùng một dòng. Nếu chúng ta không chỉ định bất kỳ kiểu nào cho widget Text, nó sẽ sử dụng kiểu lớp DefaultTextStyle gần nhất.

Đây là một ví dụ đơn giản để hiểu widget này. Ví dụ này hiển thị tiêu đề dự án của chúng ta trong thanh ứng dụng và một thông báo trong nội dung ứng dụng.

```
1 void main() {
2   runApp(MyApp());
3 }
4
5 class MyApp extends StatelessWidget {
6   @override
7   Widget build(BuildContext context) {
8     return MaterialApp(
9       theme: ThemeData(
10        primarySwatch: Colors.green,
11      ),
12      home: MyTextPage());
13   }
14 }
15
16 class MyTextPage extends StatelessWidget {
17   @override
18   Widget build(BuildContext context) {
19     return Scaffold(
20       appBar: AppBar(
21         title: Text("Text Widget Example"),
22       ),
23       body: Center(child: Text("Welcome to NgocTienTNT")),
24     );
25   }
26 }
```

Trong đoạn code trên, chúng ta đã sử dụng widget `MaterialApp` gọi màn hình chính bằng cách sử dụng lớp `MyTextPage()`. Lớp này chứa các scaffold, trong đó có `AppBar` và `Body`, nơi chúng ta đã sử dụng widget `Text` để hiển thị tiêu đề và cơ thể, tương ứng.

Trình tạo ra một widget con.

Công cụ tạo widget Text được sử dụng để tạo giao diện tùy chỉnh cho Text của chúng ta trong Flutter:

TextAlign: Nó được sử dụng để chỉ định cách Text của chúng ta được căn chỉnh theo chiều ngang. Nó cũng kiểm soát vị trí Text.

TextDirection: Nó được sử dụng để xác định cách các giá trị textAlign kiểm soát bố cục của Text của chúng ta. Thông thường, chúng ta viết Text từ trái sang phải, nhưng chúng ta có thể thay đổi nó bằng cách sử dụng tham số này.

Overflow: Nó được sử dụng để xác định khi nào Text sẽ không vừa với không gian có sẵn. Nó có nghĩa là chúng ta đã chỉ định nhiều Text hơn không gian có sẵn.

TextScaleFactor: Nó được sử dụng để xác định tỷ lệ của Text được hiển thị bởi widget Text. Giả sử chúng ta đã chỉ định hệ số tỷ lệ Text là 1,5, thì Text của chúng ta sẽ lớn hơn 50 phần trăm so với kích thước phông chữ được chỉ định.

SoftWrap: Nó được sử dụng để xác định có hay không hiển thị tất cả nội dung widget Text khi không còn đủ dung lượng. Nếu nó là sự thật, nó sẽ hiển thị tất cả nội dung. Nếu không, nó sẽ không hiển thị tất cả nội dung.

MaxLines: Nó được sử dụng để xác định số dòng tối đa được hiển thị trong widget Text.

TextWidthBasis: Nó được sử dụng để kiểm soát cách xác định chiều rộng Text

TextHeightBehavior: Nó được sử dụng để kiểm soát cách đoạn văn xuất hiện giữa dòng đầu tiên và phần cuối của dòng cuối cùng.

Style: Đây là thuộc tính phổ biến nhất của widget con này cho phép các nhà phát triển tạo kiểu dáng cho Text của họ. Nó có thể tạo kiểu bằng cách chỉ định màu nền và nền trước, cỡ chữ, độ đậm của phông chữ, khoảng cách giữa các chữ và từ, ngôn ngữ, bóng, v.v.

Bảng mô tả các thành phần của các Style Widget Text:

Thuộc tính	Mô tả
------------	-------

foreground	Xác định màu nền trước cho Text.
background	Xác định sơn làm nền cho Text.
fontWeight	Quyết định độ dày của Text.
fontSize	Xác định kích thước của Text.
fontFamily	Được sử dụng để chỉ định kiểu chữ cho phông chữ. Đối với điều này, chúng ta cần tải xuống file kiểu chữ trong dự án của mình, sau đó giữ tệp này vào thư mục assets/font. Cuối cùng, cấu hình file pubspec.yaml để sử dụng nó trong dự án.
fontStyle	Được sử dụng để tạo kiểu cho phông chữ ở dạng in đậm hoặc nghiêng.
color	Được sử dụng để xác định màu sắc của Text.
letterSpacing	Được sử dụng để xác định khoảng cách giữa các ký tự của Text.
wordSpacing	Được sử dụng để xác định khoảng cách giữa hai từ của Text.
shadows	Được sử dụng để vẽ bên dưới Text.
decoration	Chúng ta sử dụng điều này để trang trí Text bằng cách sử dụng ba tham số: decoration, decorationColor, decorationStyle. The decoration determines the location, decorationColor specify the color, decorationStyle determine xác định hình dạng.

5.3. Widget ẩn

Các widget vô hình có liên quan đến cách bố trí và kiểm soát các widget. Nó cung cấp việc kiểm soát cách các widget thực sự hoạt động và cách chúng sẽ hiển thị trên màn hình. Một số loại widget quan trọng là:

Column

Widget này sắp xếp các con của nó theo hướng dọc trên màn hình . Nói cách khác, nó sẽ mong đợi một mảng dọc gồm các widget con. Nếu widget cần lấp đầy không gian dọc có sẵn, chúng ta phải bọc các widget trong widget Mở rộng.

Một widget dạng cột không thể cuộn được vì nó hiển thị các widget trong chế độ xem hiển thị đầy đủ. Vì vậy, sẽ được coi là sai nếu chúng ta có nhiều con hơn trong một cột sẽ không phù hợp với không gian có sẵn. Nếu chúng ta muốn tạo một danh sách các widget cột có thể cuộn được, chúng ta cần sử dụng ListView Widget.

Chúng ta cũng có thể kiểm soát cách một widget cột sắp xếp các con của nó bằng cách sử dụng thuộc tính `mainAxisAlignment` và `crossAxisAlignment`. Trục chéo của cột sẽ chạy theo chiều ngang và trục chính sẽ chạy theo chiều dọc. Hình ảnh minh họa dưới đây giải thích rõ ràng hơn.



Hình 1.7: Widget Cột

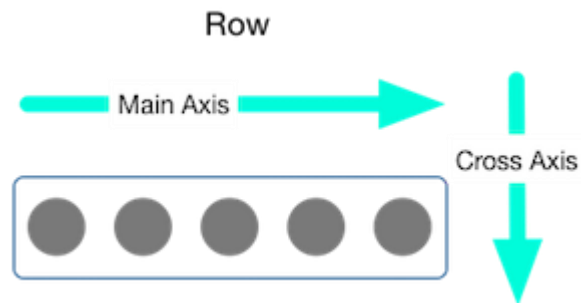
Lưu ý: widget cột cũng căn chỉnh nội dung của nó bằng cách sử dụng các thuộc tính tương tự như chúng ta đã thảo luận trong widget hàng như bắt đầu, kết thúc, giữa, khoảng trắng, khoảng trắng và khoảng trắng.

Row

Widget này sắp xếp các con của nó theo hướng nằm ngang trên màn hình. Nói cách khác, nó sẽ mong đợi các widget con trong một mảng ngang. Nếu widget cần lấp đầy không gian ngang có sẵn, chúng ta phải bọc các widget trong widget Mở rộng.

Widget hàng có vẻ không cuộn được vì nó hiển thị các widget trong chế độ xem và hiển thị toàn bộ. Vì vậy, sẽ được coi là sai nếu chúng ta có nhiều widget con hơn trong một hàng sẽ không phù hợp với không gian có sẵn. Nếu chúng ta muốn tạo một danh sách các widget hàng có thể cuộn được, chúng ta cần sử dụng widget `ListView`.

Chúng ta có thể kiểm soát cách một widget hàng căn chỉnh các con của nó dựa trên lựa chọn của chúng tôi bằng cách sử dụng thuộc tính `crossAxisAlignment` và `mainAxisAlignment`. Trục chéo của hàng sẽ chạy theo chiều dọc và trục chính sẽ chạy theo chiều ngang. Xem hình ảnh minh họa bên dưới để hiểu rõ hơn.



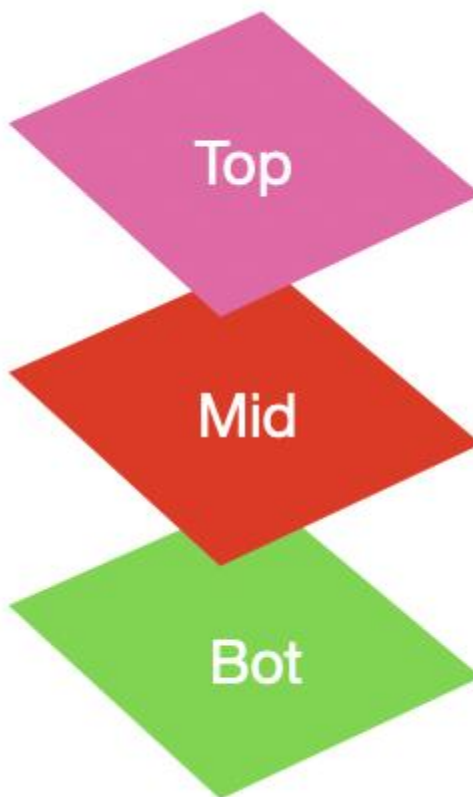
Hình 1.8: Widget Hàng

Center

Widget con này được sử dụng để căn giữa widget con, nằm bên trong nó. Tất cả các ví dụ trước đều chứa bên trong widget Center.

Stack

Ngăn xếp (Stack) là một widget trong Flutter chứa danh sách các widget và đặt chúng trên đầu các widget khác. Nói cách khác, ngăn xếp cho phép các nhà phát triển chồng nhiều widget vào một màn hình duy nhất và hiển thị chúng từ dưới lên trên. Do đó, widget đầu tiên là mục dưới cùng và widget cuối cùng là mục trên cùng.



Hình 1.9: Widget Ngăn Xếp

Các điểm chính liên quan đến Stack Widget

Sau đây là những điểm chính của widget ngăn xếp Flutter:

- Widget trong ngăn xếp có thể được định vị hoặc không được định vị.
- Các mục được định vị được bao bọc trong widget được Định vị và phải có một thuộc tính không rỗng.
- Các widget con không định vị được tự căn chỉnh. Nó hiển thị trên màn hình dựa trên sự liên kết của ngăn xếp. Vị trí mặc định của các con là ở góc trên cùng bên trái.
- Chúng ta có thể sử dụng thuộc tính alignment để thay đổi sự liên kết của các widget.
- Stack đặt các widget con theo thứ tự với con đầu tiên ở dưới cùng và con cuối cùng ở trên cùng. Nếu chúng ta muốn sắp xếp lại widget

dành cho trẻ em, thì bắt buộc phải xây dựng lại ngăn xếp theo thứ tự mới. Theo mặc định, widget đầu tiên của mỗi ngăn xếp có kích thước tối đa so với các widget khác.

Cách sử dụng một ngăn xếp (stack):



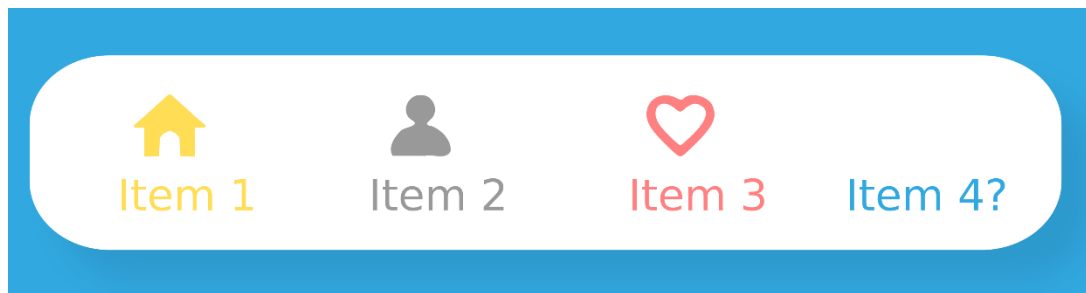
```
1  child: Stack(  
2    fit: StackFit.passthrough,  
3    children: [  
4      Container(  
5        height: 300,  
6        width: 400,  
7        color: Colors.green,  
8        child: Center(  
9          child: Text(  
10           "Bot Widget: Green",  
11           style: TextStyle(  
12             color: Colors.white,  
13             fontSize: 20,  
14           ),  
15         ),  
16       ),  
17     ),  
18     Container(  
19       height: 200,  
20       width: 300,  
21       color: Colors.red,  
22       child: Center(  
23         child: Text(  
24           "Mid Widget: Red",  
25           style: TextStyle(  
26             color: Colors.white,  
27             fontSize: 20,  
28         ),  
29       ),  
30     ),  
31   ),  
32   Container(  
33     height: 100,  
34     width: 200,  
35     color: Colors.pink,  
36     child: Center(  
37       child: Text(  
38         "Top Widget: Pink",  
39         style: TextStyle(  
40           color: Colors.white,  
41           fontSize: 20,  
42       ),  
43     ),  
44   ),  
45 )  
46 ]),
```

6. Tìm hiểu bố cục (layout) Trong giao diện Flutter

6.1. Bố cục

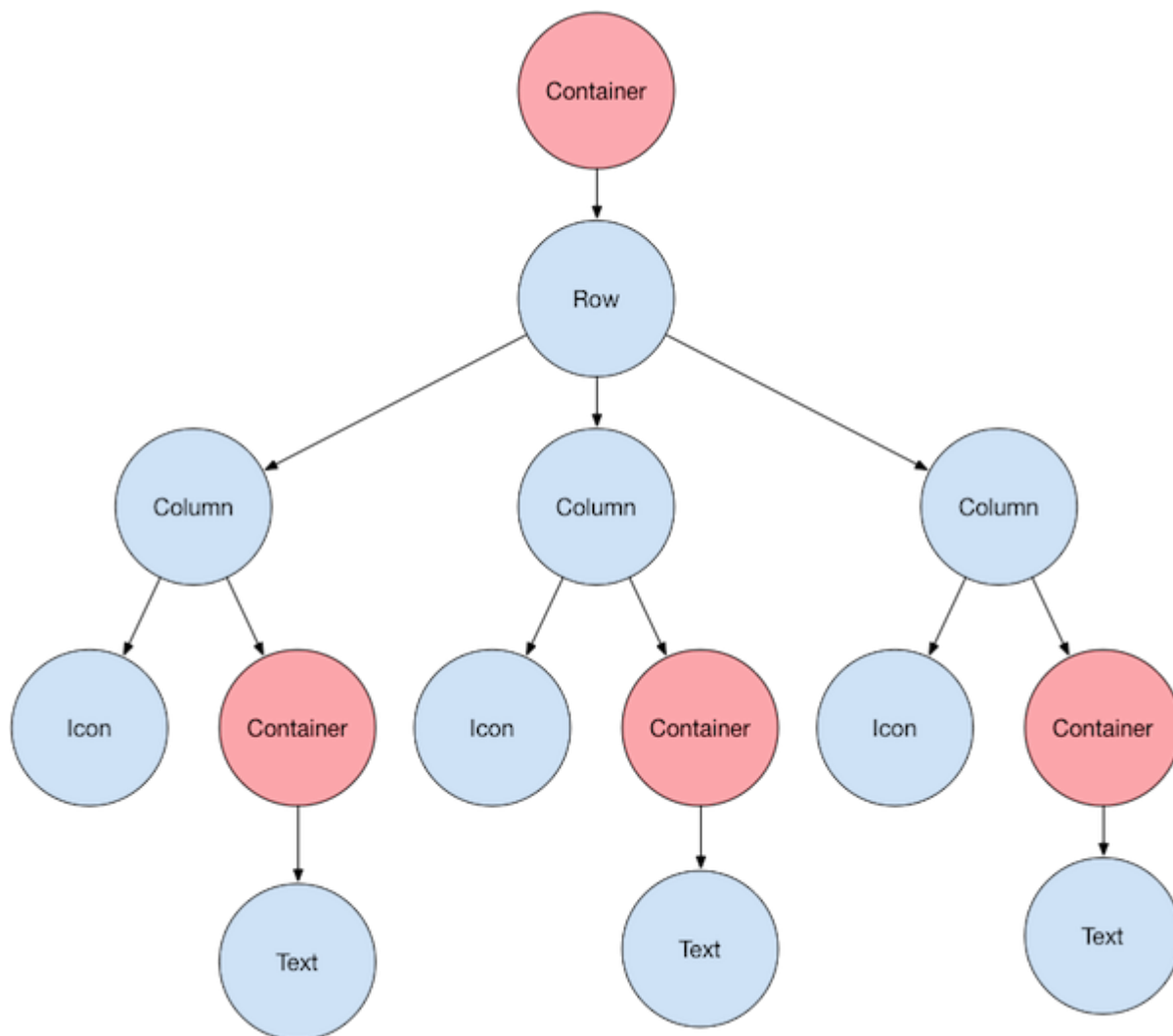
Khái niệm chính của cơ chế bố trí là widget. Chúng ta biết rằng sự flutter giả định mọi thứ như một widget. Vì vậy, hình ảnh, biểu tượng, văn bản và thậm chí cả bố cục(layout) của ứng dụng của bạn đều là widget. Ở đây, một số thứ bạn không thấy trên giao diện người dùng ứng dụng của mình, chẳng hạn như các hàng, cột và lưới sắp xếp, ràng buộc và căn chỉnh các widget hiển thị cũng là các widget.

Flutter cho phép chúng ta tạo bố cục bằng cách soạn nhiều widget để xây dựng các widget phức tạp hơn. Ví dụ, chúng ta có thể thấy hình ảnh dưới đây hiển thị ba biểu tượng với nhãn bên dưới mỗi biểu tượng.



Hình 1.10: Biểu tượng và nhãn dán trong Flutter

Trong hình ảnh thứ hai, chúng ta có thể thấy bố cục trực quan của hình ảnh trên. Hình ảnh này hiển thị một hàng gồm ba cột và các cột này chứa một biểu tượng và nhãn.



1.11: Nội dung của Container

Trong hình trên, vùng chứa là một lớp widget cho phép chúng ta tùy chỉnh widget con. Nó chủ yếu được sử dụng để thêm đường viền, đệm, lề, màu nền và nhiều thứ khác. Tại đây, widget văn bản nằm dưới vùng chứa để thêm lề. Toàn bộ hàng cũng được đặt trong một vùng chứa để thêm lề và phân đệm xung quanh hàng. Ngoài ra, phần còn lại của giao diện người dùng được kiểm soát bởi các thuộc tính như màu sắc, kiểu văn bản, v.v.

6.2. Bố cục một Widget

Hãy để chúng ta tìm hiểu cách chúng ta có thể tạo và hiển thị một widget đơn giản. Các bước sau đây cho biết cách bố trí widget:

Bước 1: Đầu tiên, bạn cần chọn một Bố cục widget.

Bước 2: Tiếp theo, tạo một widget hiển thị.

Bước 3: Sau đó, thêm widget hiển thị vào widget layout.

Bước 4: Cuối cùng, thêm widget bố cục vào trang mà bạn muốn hiển thị.

6.3. Các loại Widget bố cục

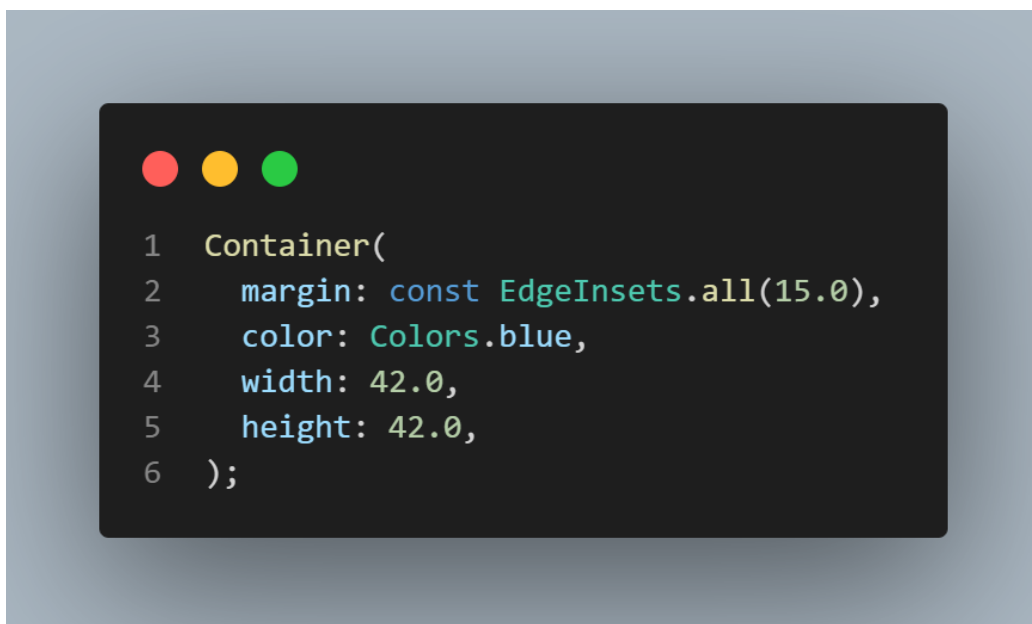
Chúng ta có thể phân loại widget bố cục thành hai loại:

- Widget đơn
- Widget đa

6.3.1. Các Widget đơn

Widget bố cục con duy nhất là một loại widget, có thể chỉ có một widget bên trong widget bố cục mẹ. Các widget này cũng có thể chứa chức năng bố cục đặc biệt. Flutter cung cấp cho chúng ta nhiều widget con để làm cho giao diện người dùng của ứng dụng trở nên hấp dẫn. Nếu chúng ta sử dụng các widget này một cách thích hợp, nó có thể tiết kiệm thời gian của chúng ta và làm cho code ứng dụng dễ đọc hơn. Danh sách các loại widget đơn lẻ khác nhau là:

Container: Đây là widget bố cục phổ biến nhất cung cấp các tùy chọn có thể tùy chỉnh để đặt màu, định vị và định cỡ các widget.



Padding: Nó là một widget được sử dụng để sắp xếp widget con của nó theo khoảng đệm đã cho. Nó chứa `EdgeInsets` và `EdgeInsets.fromLTRB` cho phía mong muốn mà bạn muốn cung cấp đệm.



```
1  Padding(  
2    padding: EdgeInsets.all(14.0),  
3    child: Text('Hello NgocTienTNT!'),  
4  );
```

Center: widget này cho phép bạn căn giữa widget trong chính nó.



```
1  Center(  
2    child: Container(  
3      margin: const EdgeInsets.all(15.0),  
4      color: Colors.blue,  
5      width: 42.0,  
6      height: 42.0,  
7    ),  
8  );
```

SizeBox: widget này cho phép bạn cung cấp kích thước được chỉ định cho widget thông qua tất cả các màn hình.



```
1  SizeBox(  
2    width: 300.0,  
3    height: 450.0,  
4    child: const Card(child: Text('Hello NgocTienTNT!')),  
5  );
```

AspectRatio: widget này cho phép bạn giữ kích thước của widget theo một tỷ lệ khung hình được chỉ định.



ConstrainedBox: Đây là một widget cho phép bạn buộc các ràng buộc bổ sung lên widget con của nó. Nó có nghĩa là bạn có thể buộc widget con có một ràng buộc cụ thể mà không làm thay đổi các thuộc tính của widget con.



6.3.2. Đa Widget

Đa widget là một loại widget chứa nhiều hơn một widget con bên trong và cách bố trí của các widget này là duy nhất. Ví dụ: widget Hàng bố trí widget theo hướng ngang và widget Cột bố trí widget theo hướng dọc. Nếu chúng ta

kết hợp widget Hàng và Cột, thì nó có thể xây dựng bất kỳ cấp độ nào của widget phức tạp.

```
1 class MyApp extends StatelessWidget {
2   // It is the root widget of your application.
3   @override
4   Widget build(BuildContext context) {
5     return MaterialApp(
6       title: 'Multiple Layout Widget',
7       debugShowCheckedModeBanner: false,
8       theme: ThemeData(
9         // This is the theme of your application.
10        primarySwatch: Colors.blue,
11      ),
12      home: MyHomePage(),
13    );
14  }
15 }
16
17 class MyHomePage extends StatelessWidget {
18   @override
19   Widget build(BuildContext context) {
20     return Center(
21       child: Container(
22         alignment: Alignment.center,
23         color: Colors.white,
24         child: Row(
25           children: [
26             Expanded(child: Text('Peter', textAlign: TextAlign.center)),
27             Expanded(child: Text('John', textAlign: TextAlign.center)),
28             Expanded(
29               child: FittedBox(
30                 fit: BoxFit.contain, // otherwise the logo will be tiny
31                 child: const FlutterLogo(),
32               ),
33             ),
34           ],
35         ),
36       ),
37     );
38   }
39 }
```

7. Tìm hiểu về Cử chỉ (Gestures) với giao diện Flutter

Cử chỉ (Gestures) là một tính năng thú vị trong Flutter cho phép chúng ta tương tác với ứng dụng di động (hoặc bất kỳ thiết bị dựa trên cảm ứng). Nói chung, cử chỉ xác định bất kỳ hành động hoặc chuyển động vật lý nào của người dùng nhằm mục đích kiểm soát thiết bị di động. Một số ví dụ về cử chỉ là:

- Khi màn hình di động bị khóa, bạn trượt ngón tay trên màn hình để mở khóa.
- Nhấn vào một nút trên màn hình điện thoại di động của bạn
- Nhấn và giữ biểu tượng ứng dụng trên thiết bị dựa trên cảm ứng để kéo biểu tượng đó qua các màn hình.

Chúng ta sử dụng tất cả những cử chỉ này trong cuộc sống hàng ngày để tương tác với điện thoại hoặc thiết bị dựa trên cảm ứng của bạn.

Flutter chia hệ thống cử chỉ thành hai lớp khác nhau, được đưa ra dưới đây:

- Con trỏ(Pointers)
- Cử chỉ(Gestures)

7.1. Con trỏ

Con trỏ(Pointers) là lớp đầu tiên đại diện cho dữ liệu thô về tương tác của người dùng. Nó có các sự kiện, mô tả vị trí và chuyển động của các con trỏ như chạm, chuột và kiểu trên màn hình. Flutter không cung cấp bất kỳ cơ chế nào để hủy hoặc dừng các sự kiện con trỏ được gửi đi thêm. Flutter cung cấp một widget Listener để lắng nghe các sự kiện con trỏ trực tiếp từ lớp widget. Con trỏ-sự kiện được phân loại thành bốn loại chủ yếu:

- PointerDownEvents
- PointerMoveEvents
- PointerUpEvents
- PointerCancelEvents

PointerDownEvents: Nó cho phép con trỏ tiếp xúc với màn hình tại một vị trí cụ thể.

PointerMoveEvents: Nó cho phép con trỏ di chuyển từ vị trí này đến vị trí khác trên màn hình.

PointerUpEvents: Nó cho phép con trỏ dừng tiếp xúc với màn hình.

PointerCancelEvents: Sự kiện này được gửi khi tương tác với con trỏ bị hủy.

7.2. Cử chỉ

Đây là lớp thứ hai đại diện cho các hành động ngữ nghĩa như chạm, kéo và chia tỷ lệ, được nhận dạng từ nhiều sự kiện con trở riêng lẻ. Nó cũng có thể gửi nhiều sự kiện tương ứng với vòng đời cử chỉ như kéo bắt đầu, kéo cập nhật và kéo kết thúc. Một số cử chỉ được sử dụng phổ biến được liệt kê dưới đây:

Chạm (Tap): Có nghĩa là chạm vào bề mặt màn hình từ đầu ngón tay trong một thời gian ngắn rồi thả chúng ra. Cử chỉ này chứa các sự kiện sau:

- onTapDown
- onTapUp
- onTap
- onTapCancel

Nhấn đúp (Double Tap): Nó tương tự như cử chỉ Nhấn, nhưng bạn cần nhấn hai lần trong thời gian ngắn. Cử chỉ này chứa các sự kiện sau:

- onDoubleTap

Kéo (Drag): Nó cho phép chúng ta chạm vào bề mặt của màn hình bằng đầu ngón tay và di chuyển nó từ vị trí này sang vị trí khác rồi thả chúng ra. Flutter phân loại kéo thành hai loại:

- Kéo ngang: Cử chỉ này cho phép con trỏ di chuyển theo hướng ngang. Nó chứa các sự kiện sau:
 - onHorizontalDragStart
 - onHorizontalDragUpdate
 - onHorizontalDragEnd
- Kéo dọc: Cử chỉ này cho phép con trỏ di chuyển theo hướng thẳng đứng. Nó chứa các sự kiện sau:
 - onVerticalDragStart
 - onVerticalDragStart
 - onVerticalDragStart

Nhấn lâu (Long Press): Có nghĩa là chạm vào bề mặt của màn hình tại một vị trí cụ thể trong một thời gian dài. Cử chỉ này chứa các sự kiện sau:

- onLongPress

Di chuyển (Pan): Có nghĩa là chạm vào bề mặt của màn hình bằng đầu ngón tay, có thể di chuyển theo bất kỳ hướng nào mà không cần nhả đầu ngón tay. Cử chỉ này chứa các sự kiện sau:

- onPanStart
- onPanUpdate
- onPanEnd

Chạm (Pinch): Có nghĩa là chạm (di chuyển ngón tay và ngón cái của một người hoặc đưa chúng lại gần nhau trên màn hình cảm ứng) bề mặt của màn hình bằng cách sử dụng hai ngón tay để phóng to hoặc thu nhỏ màn hình.

7.3. Dò cử chỉ

Flutter cung cấp một tiện ích hỗ trợ tuyệt vời cho tất cả các loại cử chỉ bằng cách sử dụng tiện ích GestureDetector. GestureWidget là các widget không trực quan, chủ yếu được sử dụng để phát hiện cử chỉ của người dùng. Ý tưởng cơ bản của bộ phát hiện cử chỉ là một tiện ích không trạng thái có chứa các tham số trong hàm tạo của nó cho các sự kiện chạm khác nhau.

Trong một số tình huống, có thể có nhiều bộ phát hiện cử chỉ tại một vị trí cụ thể trên màn hình và sau đó, khung sẽ xác định cử chỉ nào sẽ được gọi. Widget GestureDetector quyết định cử chỉ nào sẽ nhận ra dựa trên lệnh gọi lại nào của nó là không rỗng.

8. Quản lý State

Flutter cung cấp một tiện ích hỗ trợ tuyệt vời cho tất cả các loại cử chỉ bằng cách sử dụng tiện ích GestureDetector. GestureWidget là các widget không trực quan, chủ yếu được sử dụng để phát hiện cử chỉ của người dùng. Ý tưởng cơ bản của bộ phát hiện cử chỉ là một tiện ích không trạng thái có chứa các tham số trong hàm tạo của nó cho các sự kiện chạm khác nhau.

Trong một số tình huống, có thể có nhiều bộ phát hiện cử chỉ tại một vị trí cụ thể trên màn hình và sau đó, khung sẽ xác định cử chỉ nào sẽ được gọi. Widget GestureDetector quyết định cử chỉ nào sẽ nhận ra dựa trên lệnh gọi lại nào của nó là không rỗng.

State (Trạng thái)

Trạng thái là thông tin có thể đọc được khi widget được tạo và có thể thay đổi hoặc sửa đổi trong suốt thời gian tồn tại của ứng dụng. Nếu bạn muốn thay đổi widget của mình, bạn cần cập nhật đối tượng trạng thái, có thể được thực hiện bằng cách sử dụng hàm `setState()` có sẵn cho các widget `Stateful`. Hàm `setState()` cho phép chúng ta thiết lập các thuộc tính của đối tượng trạng thái kích hoạt vẽ lại giao diện người dùng.

Quản lý state (trạng thái) là một trong những quy trình phổ biến và cần thiết nhất trong vòng đời của một ứng dụng. Theo tài liệu chính thức, Flutter mang tính chất khai báo. Điều đó có nghĩa là Flutter xây dựng giao diện người dùng của mình bằng cách phản ánh trạng thái hiện tại của ứng dụng của bạn. Hình sau giải thích rõ hơn về nơi bạn có thể xây dựng giao diện người dùng từ trạng thái ứng dụng.

Trong Flutter, quản lý state (trạng thái) phân thành hai loại khái niệm, được đưa ra dưới đây:

- Trạng thái tức thời (Ephemeral State)
- Trạng thái ứng dụng (App State)

8.1. Trạng thái tức thời

Trạng thái này còn được gọi là Trạng thái giao diện người dùng hoặc trạng thái địa phương. Nó là một loại trạng thái có liên quan đến widget cụ thể, hoặc bạn có thể nói rằng nó là một trạng thái chứa trong một widget duy nhất. Trong loại trạng thái này, bạn không cần sử dụng các kỹ thuật quản lý state (trạng thái). Ví dụ phổ biến của trạng thái này là Text Field.



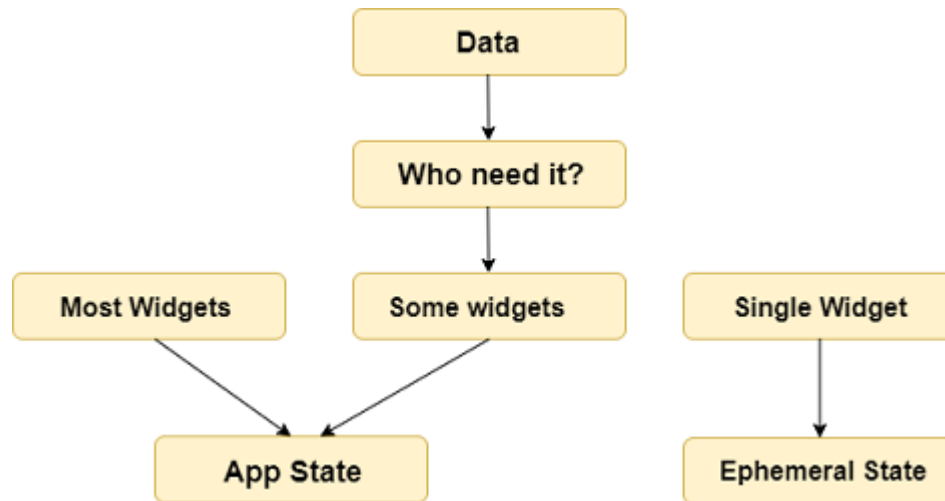
```
1 class MyHomepage extends StatefulWidget {
2   @override
3   MyHomepageState createState() => MyHomepageState();
4 }
5
6 class MyHomepageState extends State<MyHomepage> {
7   String _name = "Peter";
8   @override
9   Widget build(BuildContext context) {
10    return ElevatedButton(
11      child: Text(_name),
12      onPressed: () {
13        setState(() {
14          _name = _name == "Peter" ? "John" : "Peter";
15        });
16      },
17    );
18  }
19 }
```

Trong ví dụ trên, `_name` là một trạng thái tạm thời. Ở đây, chỉ có hàm `setState()` bên trong lớp của `StatefulWidget` mới có thể truy cập vào `_name`. Phương thức xây dựng gọi một hàm `setState()`, hàm này thực hiện sửa đổi các biến trạng thái. Khi phương thức này được thực thi, đối tượng widget sẽ được thay thế bằng đối tượng mới, mang lại giá trị biến được sửa đổi.

8.2. Trạng thái ứng dụng

Nó khác với trạng thái tức thời. Đó là một loại trạng thái mà chúng ta muốn chia sẻ trên các phần khác nhau của ứng dụng và muốn giữ lại giữa các phiên của người dùng. Do đó, loại trạng thái này có thể được sử dụng trên toàn cầu. Đôi khi nó còn được gọi là trạng thái ứng dụng hoặc trạng thái chia sẻ.

Một số ví dụ về trạng thái này là Tùy chọn người dùng, Thông tin đăng nhập, thông báo trong ứng dụng mạng xã hội, giỏ hàng trong ứng dụng thương mại điện tử, trạng thái đã đọc / chưa đọc của các bài báo trong ứng dụng tin tức, v.v.



Hình 1.12: Sự khác biệt giữa trạng thái ứng dụng và trạng thái tức thời

Ví dụ đơn giản nhất về quản lý trạng thái ứng dụng có thể được học bằng cách sử dụng gói nhà cung cấp. Việc quản lý state (trạng thái) với nhà cung cấp rất dễ hiểu và ít phải viết mã. Nhà cung cấp là thư viện của bên thứ ba. Ở đây, chúng ta cần hiểu ba khái niệm chính để sử dụng thư viện này.

1. **ChangeNotifier**: là một lớp đơn giản, cung cấp thông báo thay đổi cho người nghe của nó. Nó dễ hiểu, dễ thực hiện và được tối ưu hóa cho một số lượng nhỏ người nghe. Nó được sử dụng để người nghe quan sát một mô hình để thay đổi. Trong điều này, chúng ta chỉ sử dụng phương thức `tifyListener ()` để thông báo cho người nghe.
2. **ChangeNotifierProvider**: là widget cung cấp một phiên bản của **ChangeNotifier** cho con của nó. Nó đến từ gói nhà cung cấp.
3. **Khách hàng (Consumer)**: Nó là một loại nhà cung cấp không làm bất kỳ công việc cầu kỳ. Nó chỉ gọi nhà cung cấp trong một widget con mới và ủy quyền triển khai bản dựng của nó cho người xây dựng

9. Quản lý thư viện và các gói trong Flutter (Lib & Package)

9.1. Nạp thư viện trong Dart

Một thư viện có thể chứa tập hợp các lớp, hàm, hằng ... để bạn nạp vào và sử dụng khi cần thiết.

Trong Dart để nạp thư viện bạn dùng từ khóa `import` ở đầu file. Với cú pháp:

```
import 'uri';
```

Nếu chỉ nạp một phần nào đó của thư viện sử dụng thêm từ khóa `show` có dạng:

```
import 'uri' show part1, part2, part3;
```

Khi nạp thư viện, để tránh xung đột về tên, bạn cũng có thể đặt lại bằng `as`

Trong đó `uri` như là một định danh duy nhất trỏ đến thư viện cần nạp. Nó có thể có các dạng:

Các thư viện xây dựng sẵn của Dart có cấu trúc `uri` `dart:tên_thư_viện` ví dụ `dart:convert`, `dart:html`, `dart:math`, `dart:js`, `dart:web_sql`. Ví dụ cần nạp thư viện toán học `dart:math` thì dùng

```
import 'dart:math';
```

Nạp từ file dự án, thì `url` chỉ đến đường dẫn file dart cần nạp vào. Ví dụ bạn có file `lib/mylib.dart` trong dự án thì nạp vào dùng bằng:

```
import 'lib/mylib.dart';
```

Thư viện nạp từ các gói package tải về, thì `uri` có dạng `package:tên_gói/thư_viện_gói`. Ví dụ gói `googleapis_auth`, có thành phần `auth_browser` cung cấp chức năng xác thực Auth với tài khoản google, thì nạp thư viện đó vào bằng:

```
import "package:googleapis_auth/auth_browser.dart";
```

9.2. Các thư viện cung cấp sẵn

Những thư viện Dart đi cùng SDK Dart, bạn có thể nạp luôn bằng `import 'dart:tên_thư_viện'`, danh sách thư viện như: (chi tiết tại [LIBRARIES](#))

Thư viện	Chức năng
<code>dart:core</code>	Thư viện cung cấp các hàm, lớp cơ bản, nó tự động nạp nên bạn không cần import thư viện này

dart:collection	Cung cấp các cấu trúc dữ liệu như HashSet, HashMap, Queue, ...
dart:math	Cung cấp hàm toán học
dart:convert	Tính năng mã hóa, giải mã dữ liệu, kể cả JSON, UTF-8
dart:io	Thư viện IO cung cấp các chức năng về File, Socket, HTTP

9.3. Các gói hay sử dụng

9.3.1. Flutter Bloc

Bloc là một lib để quản lý state cho Flutter application. B.L.o.C nghĩa là Business Logic Component. Nhận 'Event' như là input và trả về output là 'State'. Bloc được xây dựng dựa trên RxDart.

Chúng ta có thể chia Flutter application architecture thành 3 lớp sau:

- View Layer (Presentation Layer): Có nhiệm vụ render chính nó dựa trên một hoặc nhiều state của bloc. Nó cũng xử lý các sự kiện input của user và lifecycle event của application.
- Bloc (Business Logic): Có nhiệm vụ nhận event từ lớp biểu diễn và trả về state mới. Hoạt động như một cầu nối giữa lớp data và lớp presentation.
- Data Layer: Có nhiệm vụ cung cấp data từ bất kể nguồn nào. Data Provider cung cấp data thô và repository sẽ là trình đóng gói một hoặc nhiều data providers.

Vậy có nghĩa là Bloc đứng giữa 2 lớp View và Data.

Một số concept cho Bloc:

1. **Events:** Event được truyền vào một Bloc. Nó giống như concept action trong Redux. Trong lớp presentation, event được tạo ra bởi tương tác của user như button click và truyền vào Bloc. Một event có thể bao gồm vài data được thêm vào.
2. **States:** State là một phần của application state. Nó là output của Bloc. Khi state thay đổi, thành phần UI sẽ được thông báo và dựa vào current state, nó sẽ tự render lại.

3. **Transition:** Thay đổi từ một state sang state khác gọi là transition. Nó bao gồm current state, event và state tiếp theo.

Thư viện Flutter Bloc: [tại đây](#)

9.3.2. Provider

Provider là loại package cơ bản nhất trong số các loại package của Provider. Bạn có thể sử dụng nó để cung cấp một giá trị (thường là một data model) cho bất kỳ vị trí nào trong widget tree. Tuy nhiên, nó sẽ không giúp bạn cập nhật widget tree khi mà giá trị đó thay đổi. Bạn có thể hình dung như việc nó chỉ set dữ liệu vào mà UI nó không có sự thay đổi gì, không nhận biết được sự thay đổi.

Để có thể cung cấp data model đó cho widget tree bạn cần bọc phần trên cùng của widget tree bằng một Provider. Với tham số truyền vào là data model vừa tạo. Trong widget tree để có thể tham chiếu đến data model, cần sử dụng Consumer widget để lấy ra data model vừa truyền vào ở trên.

Một số thành phần cơ bản của Provider:

- Provider
- ChangeNotifierProvider
- ValueListenableProvider
- StreamProvider
- FutureProvider
- MultiProvider
- ProxyProvider
- ChangeNotifierProxyProvider

ChangeNotifierProvider

Không giống như Provider, ChangeNotifierProvider lắng nghe các thay đổi trong data model. Khi có thay đổi, nó sẽ xây dựng lại bất kỳ widget nào trong Consumer.

Trong hàm build thay đổi Provider thành ChangeNotifierProvider. Lớp mô hình cần sử dụng extend ChangeNotifier (hoặc with ChangeNotifier). Điều này cung cấp cho bạn quyền truy cập vào notifyListeners() và bất kỳ lúc nào bạn gọi notifyListeners() thì ChangeNotifierProvider sẽ được thông báo và tất cả các widget bên trong Consumers sẽ được rebuild lại.

FutureProvider

FutureProvider về cơ bản chỉ là một wrapper với bên trong là FutureBuilder. Bạn cung cấp cho nó một số dữ liệu ban đầu để hiển thị trong giao diện người dùng và cũng có thể cung cấp cho nó một hoạt động bất đồng bộ Future của giá trị mà bạn muốn cung cấp. FutureProvider lắng nghe khi Future hoàn thành và sau đó thông báo cho Consumers để xây dựng lại các widget của nó.

StreamProvider

StreamProvider về cơ bản là một wrapper với bên trong là một StreamBuilder. Bạn cung cấp một Stream và sau đó Consumer được xây dựng lại khi có sự kiện trong stream. Thiết lập rất giống với FutureProvider ở trên.

ValueListenableProvider

Nó giống như ChangeNotifierProvider nhưng phức tạp hơn và không có bất kỳ giá trị gia tăng rõ ràng nào.

MultiProvider

Nếu bạn cần cung cấp từ 2 loại model object trở lên, bạn có thể lồng các provider Tuy nhiên, có một cách khác gọn gàng hơn là sử dụng MultiProvider.

ProxyProvider

Điều gì sẽ xảy ra nếu bạn có hai Model mà bạn muốn cung cấp, nhưng một trong các Model phụ thuộc vào mô hình còn lại? Trong trường hợp đó, bạn có thể sử dụng ProxyProvider. ProxyProvider lấy giá trị từ một provider và cho phép nó được đưa vào provider khác.

ProxyProvider cơ bản có hai loại model trong dấu ngoặc nhọn. Model thứ hai thì phụ thuộc vào model đầu tiên.

Thư viện Provider: [tại đây](#)

9.3.3. Get

Get làm được những gì:

- State Manager : Quản lý state trong Flutter
- Navigation Manager: Quản lý việc điều hướng
- Dependencies Manager: Cung cấp giải pháp dependencies injection tuyệt vời

- Utils function: Các hàm tiện ích cực kỳ hữu ích trong lập trình Flutter

Get - State manager

Những ưu điểm của Get - State manager

- Chỉ update những widget cần thiết
- Sử dụng ít bộ nhớ hơn so với các kiểu quản lý state khác
- Quên đi StatefulWidget. Với Get các bạn không phải suy nghĩ sử dụng StatefulWidget hay StatelessWidget nữa. Bây giờ bạn chỉ việc một component duy nhất là GetWidget
- Việc tổ chức cấu trúc project sẽ cực kỳ clear, phần code logic được tách hẳn hoàn toàn so với UI (mình sẽ demo structure dự án ở bài cuối của seri)
- Update widgets without spending ram for that
- Tối ưu hoá bộ nhớ, bạn sẽ không phải lo lắng việc Out Memory nữa, Get sẽ tự động thu dọn những component không cần thiết

Get Route manager

Để sử dụng được Get Route manager bạn cũng cần phải sử dụng GetMaterialApp thay vì MaterialApp như thông thường

Thư viện Get: [tại đây](#)

9.3.4. SQFLite

SQLite là một thư viện phần mềm cơ sở dữ liệu phổ biến cung cấp hệ thống quản lý cơ sở dữ liệu quan hệ để lưu trữ cục bộ / máy khách. Nó là một công cụ cơ sở dữ liệu nhẹ và được kiểm tra theo thời gian và chứa các tính năng như công cụ cơ sở dữ liệu SQL giao dịch độc lập, không cần máy chủ, không cấu hình.

Flutter SDK không hỗ trợ SQLite trực tiếp. Thay vào đó, nó cung cấp một plugin sqflite, thực hiện tất cả các hoạt động trên cơ sở dữ liệu tương tự như thư viện SQLite. Sqflite cung cấp hầu hết các chức năng cốt lõi liên quan đến cơ sở dữ liệu như sau:

- Nó tạo hoặc mở cơ sở dữ liệu SQLite.
- Nó có thể thực thi các câu lệnh SQL một cách dễ dàng.

- Nó cũng cung cấp một phương pháp truy vấn nâng cao để lấy thông tin từ cơ sở dữ liệu SQLite.

Thư viện SQLite: [tại đây](#)

9.3.5. Shared Preferences

Shared Preferences là nơi bạn có thể lưu trữ các thông tin dưới dạng key-value được xây dựng sẵn trong hệ điều hành Android.

Thư viện Shared Preferences: [tại đây](#)

9.3.6. Http

Flutter cung cấp cho ta package http để sử dụng các phương thức HTTP. Http là một thư viện dựa trên Future và sử dụng các chức năng await và async. Nó cung cấp nhiều phương thức cấp cao và đơn giản hóa việc phát triển của ứng dụng di động dựa trên REST.

Thư viện Http: [tại đây](#)

9.3.7. Url Launcher

Url Launcher được sử dụng để mở trang web, tạo email, mở bản đồ,... Url Launcher sẽ giúp bạn tương tác với các ứng dụng bên ngoài.

Thư viện Url Launcher: [tại đây](#)

9.3.8. Font Awesome Flutter

Thư viện này cung cấp cho bạn rất nhiều icon đẹp từ fontawesome.com

Thư viện Font Awesome Flutter: [tại đây](#)

10. Tìm hiểu về Navigator và Routing trong Flutter

Điều hướng và định tuyến (Navigation and Routing) là một số khái niệm cốt lõi của tất cả các ứng dụng di động, cho phép người dùng di chuyển giữa các trang khác nhau. Chúng ta biết rằng mọi ứng dụng di động đều chứa một số màn hình để hiển thị các loại thông tin khác nhau. Ví dụ: một ứng dụng có thể có màn hình chứa nhiều sản phẩm khác nhau. Khi người dùng chạm vào sản phẩm đó, ngay lập tức nó sẽ hiển thị thông tin chi tiết về sản phẩm đó.

Trong Flutter, các màn hình và trang được gọi là các tuyến và các tuyến này chỉ là một widget. Trong Android, một tuyến tương tự như Activity, trong khi trong iOS, nó tương đương với ViewController.

Trong bất kỳ ứng dụng di động nào, điều hướng đến các trang khác nhau xác định quy trình làm việc của ứng dụng và cách xử lý điều hướng được gọi là định tuyến. Flutter cung cấp một lớp định tuyến cơ bản `MaterialPageRoute` và hai phương thức `Navigator.push()` và `Navigator.pop()` cho biết cách điều hướng giữa hai tuyến đường. Các bước sau là bắt buộc để bắt đầu điều hướng trong ứng dụng của bạn.

Bước 1: Đầu tiên, bạn cần tạo hai tuyến đường.

Bước 2: Sau đó, điều hướng đến một tuyến đường từ một tuyến đường khác bằng cách sử dụng phương thức `Navigator.push()`.

Bước 3: Cuối cùng, điều hướng đến tuyến đường đầu tiên bằng cách sử dụng phương thức `Navigator.pop()`.

10.1. Tạo routes

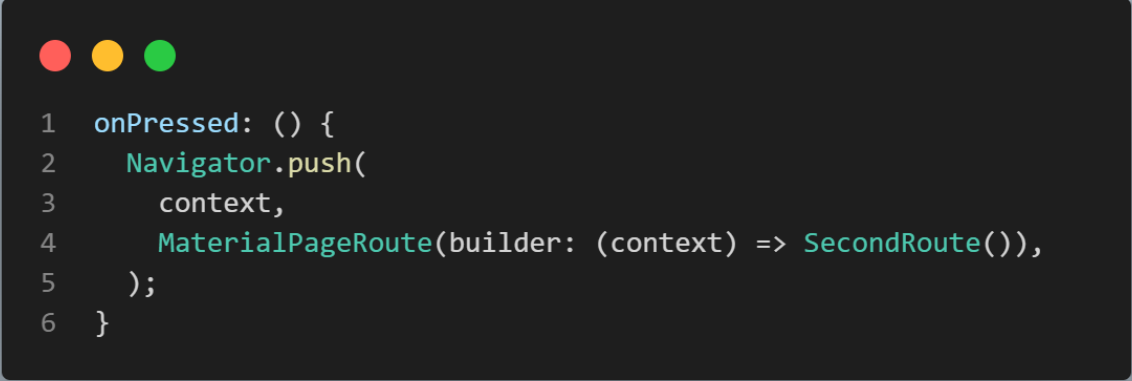
Ở đây, chúng ta sẽ tạo hai tuyến đường để điều hướng. Trong cả hai tuyến đường, chúng ta chỉ tạo một nút duy nhất. Khi chúng ta nhấn vào nút trên trang đầu tiên, nó sẽ điều hướng đến trang thứ hai. Một lần nữa, khi chúng ta nhấn vào nút trên trang thứ hai, nó sẽ trở lại trang đầu tiên. Đoạn mã dưới đây tạo ra hai tuyến đường trong ứng dụng Flutter.



```
1  class FirstRoute extends StatelessWidget {
2    @override
3    Widget build(BuildContext context) {
4      return Scaffold(
5        appBar: AppBar(
6          title: Text('First Route'),
7        ),
8        body: Center(
9          child: ElevatedButton(
10             child: Text('Open route'),
11             onPressed: () {
12               // Navigate to second route when tapped.
13             },
14          ),
15        ),
16      );
17    }
18  }
19
20  class SecondRoute extends StatelessWidget {
21    @override
22    Widget build(BuildContext context) {
23      return Scaffold(
24        appBar: AppBar(
25          title: Text("Second Route"),
26        ),
27        body: Center(
28          child: ElevatedButton(
29            onPressed: () {
30              // Navigate back to first route when tapped.
31            },
32            child: Text('Go back!'),
33          ),
34        ),
35      );
36    }
37  }
```

10.2. Điều hướng sang route thứ hai bằng phương thức Navigator.push()

Phương thức Navigator.push() được sử dụng để điều hướng / chuyển sang một tuyến đường / trang / màn hình mới. Ở đây, phương thức push () thêm một trang / tuyến đường trên ngăn xếp và sau đó quản lý nó bằng cách sử dụng Bộ điều hướng. Một lần nữa, chúng ta sử dụng lớp MaterialPageRoute cho phép chuyển đổi giữa các tuyến bằng cách sử dụng hoạt ảnh dành riêng cho nền tảng. Đoạn mã dưới đây giải thích việc sử dụng phương thức Navigator.push().



```
1  onPressed: () {  
2    Navigator.push(  
3      context,  
4      MaterialPageRoute(builder: (context) => SecondRoute()),  
5    );  
6  }
```

10.3. Quay lại route đầu tiên bằng phương thức Navigator.pop()

Bây giờ, chúng ta cần sử dụng phương thức Navigator.pop () để đóng tuyến thứ hai và quay lại tuyến đầu tiên. Phương thức pop() cho phép chúng ta loại bỏ tuyến đường hiện tại khỏi ngăn xếp, được quản lý bởi Bộ điều hướng.

Để triển khai quay lại tuyến ban đầu, chúng ta cần cập nhật phương thức gọi lại onPressed () trong tiện ích con SecondRoute như đoạn mã bên dưới:



10.4. Điều hướng với các route được đặt tên

Chúng ta đã biết cách điều hướng đến một màn hình mới bằng cách tạo một tuyến đường mới và quản lý nó bằng cách sử dụng Bộ điều hướng. Bộ điều hướng duy trì lịch sử dựa trên ngăn xếp của các tuyến đường. Nếu có nhu cầu điều hướng đến cùng một màn hình trong nhiều phần của ứng dụng, thì cách làm này không có lợi vì nó dẫn đến trùng lặp mã. Giải pháp cho vấn đề này có thể được loại bỏ bằng cách xác định các tuyến đường được đặt tên và có thể sử dụng các tuyến đường được đặt tên để điều hướng.

Chúng ta có thể làm việc với các tuyến đường được đặt tên bằng cách sử dụng hàm `Navigator.pushNamed()`. Hàm này nhận hai đối số bắt buộc (xây dựng ngữ cảnh và chuỗi) và một đối số tùy chọn. Ngoài ra, chúng ta biết về `MaterialPageRoute`, chịu trách nhiệm chuyển đổi trang. Nếu chúng ta không sử dụng điều này, thì rất khó để thay đổi trang.

Các bước sau là cần thiết, trình bày cách sử dụng các tuyến đường được đặt tên.

Bước 1: Đầu tiên, chúng ta cần tạo hai màn hình. Đoạn mã sau tạo hai màn hình trong ứng dụng của chúng ta.



```
1  class HomeScreen extends StatelessWidget {
2    @override
3    Widget build(BuildContext context) {
4      return Scaffold(
5        appBar: AppBar(
6          title: Text('Home Screen'),
7        ),
8        body: Center(
9          child: ElevatedButton(
10             child: Text('Click Here'),
11             onPressed: () {
12               //
13             },
14          ),
15        ),
16      );
17    }
18  }
19
20  class SecondScreen extends StatelessWidget {
21    @override
22    Widget build(BuildContext context) {
23      return Scaffold(
24        appBar: AppBar(
25          title: Text("Second Screen"),
26        ),
27        body: Center(
28          child: ElevatedButton(
29             onPressed: () {
30               //
31             },
32             child: Text('Go back!'),
33          ),
34        ),
35      );
36    }
37  }
```


Bước 2: Xác định các tuyến đường(routes).

Trong bước này, chúng ta phải xác định các tuyến đường. Phương thức khởi tạo `MaterialApp` chịu trách nhiệm xác định tuyến đường ban đầu và các tuyến đường khác. Tại đây, tuyến đường ban đầu cho biết phần bắt đầu của trang và thuộc tính tuyến đường xác định các tuyến đường và tiện ích con được đặt tên có sẵn. Đoạn mã sau đây giải thích rõ ràng hơn.

```
1  MaterialApp(  
2    title: 'Named Route Navigation',  
3    theme: ThemeData(  
4      // This is the theme of your application.  
5      primarySwatch: Colors.green,  
6    ),  
7    // It start the app with the "/" named route. In this case, the app starts  
8    // on the HomeScreen widget.  
9    initialRoute: '/',  
10   routes: {  
11     // When navigating to the "/" route, build the HomeScreen widget.  
12     '/': (context) => HomeScreen(),  
13     // When navigating to the "/second" route, build the SecondScreen widget.  
14     '/second': (context) => SecondScreen(),  
15   },  
16 );
```

Bước 3: Điều hướng đến màn hình thứ hai bằng chức năng `Navigator.pushNamed()`.

Trong bước này, chúng ta cần gọi phương thức `Navigator.pushNamed()` để điều hướng. Đối với điều này, chúng ta cần cập nhật một lệnh gọi lại `onPressed()` trong phương thức xây dựng của Màn hình chính như các đoạn mã bên dưới.

```
1 onPressed: () {  
2   // Navigate to the second screen by using the named route.  
3   Navigator.pushNamed(context, '/second');  
4 }
```

Bước 4: Sử dụng hàm `Navigator.pop()` để quay lại màn hình đầu tiên.

Đây là bước cuối cùng, nơi chúng ta sẽ sử dụng phương thức `Navigator.pop()` để quay lại màn hình đầu tiên.

```
1 onPressed: () {  
2   Navigator.pop(context);  
3 }
```

11. Tìm hiểu về Database trong Flutter

Cơ sở dữ liệu (Database) là một tập hợp dữ liệu có tổ chức, hỗ trợ việc lưu trữ và thao tác dữ liệu và được truy cập điện tử từ hệ thống máy tính. Chúng ta có thể tổ chức dữ liệu thành các hàng, cột, bảng và chỉ mục. Nó giúp cho việc quản lý dữ liệu trở nên dễ dàng. Chúng tôi có thể lưu trữ nhiều thứ trong cơ sở dữ liệu, như tên, tuổi, hình ảnh, hình ảnh, tệp, pdf, v.v.

Flutter cung cấp nhiều gói để làm việc với cơ sở dữ liệu. Các gói phổ biến và được sử dụng nhiều nhất là:

1. **Cơ sở dữ liệu SQLite:** Nó cho phép truy cập và thao tác với cơ sở dữ liệu SQLite.
2. **Cơ sở dữ liệu Firebase:** Nó sẽ cho phép bạn truy cập và thao tác với cơ sở dữ liệu đám mây

11.1. Cơ sở dữ liệu SQLite

SQLite là một thư viện phần mềm cơ sở dữ liệu phổ biến cung cấp hệ thống quản lý cơ sở dữ liệu quan hệ để lưu trữ cục bộ / máy khách. Nó là một công cụ cơ sở dữ liệu nhẹ và được kiểm tra theo thời gian và chứa các tính năng như công cụ cơ sở dữ liệu SQL giao dịch độc lập, không cần máy chủ, không cấu hình.

Flutter SDK không hỗ trợ SQLite trực tiếp. Thay vào đó, nó cung cấp một plugin sqflite, thực hiện tất cả các hoạt động trên cơ sở dữ liệu tương tự như thư viện SQLite. Sqflite cung cấp hầu hết các chức năng cốt lõi liên quan đến cơ sở dữ liệu như sau:

- Nó tạo hoặc mở cơ sở dữ liệu SQLite.
- Nó có thể thực thi các câu lệnh SQL một cách dễ dàng.
- Nó cũng cung cấp một phương pháp truy vấn nâng cao để lấy thông tin từ cơ sở dữ liệu SQLite.

Các bước lưu trữ, tìm và lưu dữ liệu

Bước 1: Đầu tiên, tạo một dự án mới trong Android Studio và thêm các phần phụ thuộc vào tệp pubspec.yaml.

```
1 dependencies:
2   flutter:
3     sdk: flutter
4
5   sqflite: any
6   path_provider: any
```

Gói sqflite cung cấp các lớp và chức năng để tương tác với cơ sở dữ liệu SQLite.

Các path_provider gói cung cấp các chức năng để xác định vị trí của cơ sở dữ liệu của bạn trên hệ thống địa phương, chẳng hạn như TemporaryDirectory và ApplicationDocumentsDirectory .

Bước 2: Tạo một lớp mô hình. Ở bước này, chúng ta phải xác định dữ liệu cần lưu trữ trước khi tạo bảng để lưu trữ thông tin. Đoạn mã sau đây giải thích nó một cách dễ dàng.

```
1 class Book {
2   final int id;
3   final String title;
4   final int price;
5   Book({required this.id, required this.title, required this.price});
6 }
```

Bước 3: Mở cơ sở dữ liệu. Tại đây, chúng ta cần mở kết nối với cơ sở dữ liệu. Nó yêu cầu hai bước:

- Đặt đường dẫn đến cơ sở dữ liệu bằng cách sử dụng phương thức `getDatabasePath()` và kết hợp nó với gói đường dẫn.
- Sử dụng hàm `openDatabase()` để mở cơ sở dữ liệu.

```
1 final Future<Database> database = openDatabase(  
2     join(await getDatabasesPath(), 'book_database.db'),  
3 );
```

Bước 4: Tạo bảng. Trong bước này, chúng ta phải tạo một bảng lưu trữ thông tin về các cuốn book. Ở đây, chúng ta sẽ tạo một bảng có tên book, trong đó có id, tên book và giá của book. Chúng được thể hiện dưới dạng ba cột trong bảng book.

```
1 final Future<Database> database = openDatabase(  
2     join(await getDatabasesPath(), 'book_database.db'),  
3     // When you create a database, it also needs to create a table to store books.  
4     onCreate: (db, version) {  
5         return db.execute(  
6             "CREATE TABLE books(id INTEGER PRIMARY KEY, title TEXT, price INTEGER)",  
7         );  
8     },  
9     version: 1,  
10 );
```

Bước 5: Chèn book vào cơ sở dữ liệu. Ở đây, bạn phải lưu trữ thông tin trên bảng về các cuốn book khác nhau. Chèn một cuốn book vào bảng bao gồm hai bước:

- Chuyển book thành map
- Sử dụng phương thức `insert()`

```
1 class Book {
2     final int id;
3     final String title;
4     final int price;
5     Book({required this.id, required this.title, required this.price});
6
7     /* It converts a Book into a Map. The keys correspond to
8        the names of the columns in the database.*/
9     Map<String, dynamic> toMap() {
10         return {'id': id, 'title': title, 'price': price};
11     }
12
13     Future<void> insertBook(Book book) async {
14         final Database db = await database;
15         await db.insert(
16             'books',
17             book.toMap(),
18             conflictAlgorithm: ConflictAlgorithm.replace,
19         );
20     }
21 }
```

Bước 6: Lấy danh sách book. Bây giờ, chúng ta đã lưu trữ book vào cơ sở dữ liệu và bạn có thể sử dụng truy vấn để truy xuất một cuốn book cụ thể hoặc danh sách tất cả các cuốn book. Nó bao gồm hai bước:

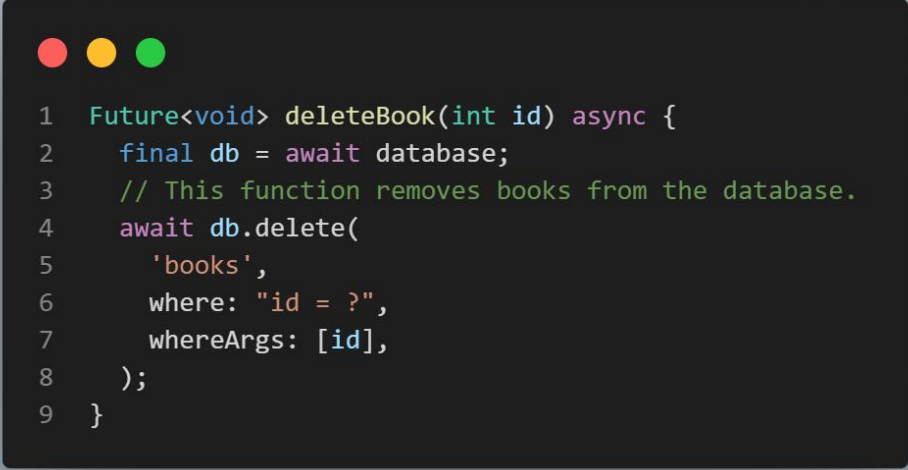
- Chạy một truy vấn trả về `List<Map>`.
- Chuyển `List<Map>` thành `List<book>`.

```
1 Future<List<Book>> books() async {
2   final Database db = await database;
3   // Use query for all Books.
4   final List<Map<String, dynamic>> maps = await db.query('maps');
5   return List.generate(maps.length, (i) {
6     return Book(
7       id: maps[i]['id'],
8       title: maps[i]['title'],
9       price: maps[i]['price'],
10    );
11  });
12 }
13
14 // It prints all the books.
15 print(await books());
```

Bước 7: Cập nhật book trong cơ sở dữ liệu. Bạn có thể sử dụng phương thức `update()` để cập nhật book mà bạn muốn. Nó bao gồm hai bước:

- Chuyển đổi book thành Bản đồ.
- Sau đó, sử dụng mệnh đề `where` để cập nhật book.

Bước 8: Xóa book khỏi cơ sở dữ liệu. Bạn có thể sử dụng phương thức `delete()` để xóa cơ sở dữ liệu. Đối với điều này, bạn cần tạo một hàm lấy id và xóa cơ sở dữ liệu của id phù hợp.



```
1 Future<void> deleteBook(int id) async {  
2   final db = await database;  
3   // This function removes books from the database.  
4   await db.delete(  
5     'books',  
6     where: "id = ?",  
7     whereArgs: [id],  
8   );  
9 }
```

Thông qua các bước làm việc với cơ sở dữ liệu SQLite thì chúng ta có thể thấy được các bước thêm mới xóa sửa cơ sở dữ liệu trong Flutter.

11.2. Firebase – NoSQL - API lưu trữ online

11.2.1. Sơ lược về Firebase

FireBase là một dạng database lưu trữ theo cách truyền thống. Mọi lưu trữ data trong bộ sưu tập giống như bảng trong database truyền thống. Tài liệu được lưu trữ trong các bộ sưu tập này. Các kiểu lưu trữ data như string, int, ... Chúng cũng có thể được liên kết đến những tài liệu khác. Mặc dù FireBase không liên kết hoàn toàn, nhưng bạn vẫn có thể tạo được sự liên kết với tài liệu của mình.

Quy trình thiết lập cho Firebase khá liên quan so với các tùy chọn trên thiết bị khác, như Moor hoặc Hive, nhưng bạn vẫn có được sự đồng bộ hóa dữ liệu giữa máy khách và máy chủ. Điều này có nghĩa là nếu bạn có nhiều khách hàng với một ứng dụng và tất cả họ đều tương tác với cùng một dữ liệu, thì dữ liệu này có thể được giữ đồng bộ giữa các khách hàng này. Ngoài ra, thiết lập này được trình bày khá tốt trong Google Codelab tại đây. Nhược điểm duy nhất của phương pháp này là bạn không nhận được lượng dữ liệu mạnh giống như cách bạn làm với Moor hoặc Hive. Bạn sẽ phải tự làm việc này bằng tay.

Ưu điểm:

- Đồng bộ hóa với Firebase trực tuyến theo thời gian thực.
- Hỗ trợ công cụ tuyệt vời.
- Dễ dàng duyệt dữ liệu trực tuyến thông qua bảng điều khiển Firebase.

Nhược điểm :

- Thiết lập Firebase có thể phức tạp nếu bạn đã thêm nó vào ứng dụng của mình.
- Vì cơ sở dữ liệu đang trực tuyến, bạn cần chú ý nhiều hơn về cơ sở dữ liệu trên thiết bị (ví dụ như quyền truy cập).

11.2.2. Giao tiếp API trong hệ thống

Bên cạnh việc lưu trữ cục bộ bằng SQLite, ứng dụng còn có thể mở rộng khả năng giao tiếp và xử lý dữ liệu thông qua API (Application Programming Interface). API cho phép ứng dụng gửi và nhận dữ liệu từ các máy chủ hoặc dịch vụ bên ngoài một cách có tổ chức và an toàn.

Trong môi trường Flutter, các thư viện phổ biến như http, dio hoặc retrofit thường được sử dụng để tương tác với API. Các thao tác thường gặp bao gồm:

- Gửi yêu cầu HTTP (GET, POST, PUT, DELETE) đến máy chủ.
- Nhận và phân tích dữ liệu JSON phản hồi.
- Chuyển đổi dữ liệu JSON thành mô hình Dart (data model).
- Xử lý trạng thái bất đồng bộ với Future, async/await hoặc sử dụng các thư viện như Provider, Bloc để quản lý logic ứng dụng.

API giúp mở rộng ứng dụng bằng các tính năng như:

- Đồng bộ dữ liệu người dùng với máy chủ.
- Phân tích và thống kê chi tiêu.
- Gợi ý chi tiêu thông minh dựa trên hành vi người dùng.
- Sao lưu và phục hồi dữ liệu đám mây qua Firebase hoặc các dịch vụ khác.

11.2.3. Thêm Firebase vào ứng dụng Flutter của bạn

Điều kiện tiên quyết

Cài đặt trình soạn thảo hoặc IDE ưa thích của bạn.

Thiết lập thiết bị hoặc trình mô phỏng để chạy ứng dụng của bạn. Trình giả lập phải sử dụng hình ảnh giả lập với Google Play.

Đảm bảo rằng ứng dụng của bạn đáp ứng các yêu cầu sau:

- API mục tiêu cấp 19 (KitKat) trở lên
- Sử dụng Android 4.4 trở lên

Cài đặt Flutter cho hệ điều hành cụ thể của bạn, bao gồm:

- SDK Flutter
- Thư viện hỗ trợ
- SDK và phần mềm dành riêng cho nền tảng
- Đăng nhập vào Firebase bằng tài khoản Google của bạn.

Nếu bạn chưa có ứng dụng Flutter, bạn có thể hoàn thành Get Started: Test Drive để tạo ứng dụng Flutter mới bằng trình chỉnh sửa hoặc IDE ưa thích của bạn.

CHƯƠNG 3: XÂY DỰNG ỨNG DỤNG

1. Tổng quan

1.1. Tên ứng dụng

Phần mềm quản lý chi tiêu cá nhân trên di động

1.2. Lý do chọn ứng dụng

Hiện nay thiết bị di động – smart phone ngày càng phổ biến. Việc đưa một ứng dụng lên các kho lưu trữ tương đối dễ dàng. Hơn nữa, người sử dụng cũng sẵn sàng chi trả một khoảng tiền phù hợp cho những ứng dụng mà họ cảm thấy cần thiết hơn so với trước đây.

Vấn đề cân đối giữa thu-cho, khoản vay nợ và kỳ hạn phải trả luôn là một vấn đề khó khăn với đa phần tầng lớp trong xã hội. Bởi vậy, một ứng dụng chạy trên thiết bị di động và giúp đỡ người sử dụng trong việc quản lý chi tiêu là rất cần thiết.

Một số phần mềm có tính năng tương tự đã được phát triển trên các nền tảng Android, IOS, ... như: Money lover, Sổ thu chi, Sổ thu chi misa, ..., tuy nhiên các phần mềm này còn nhiều bất lợi cho người dùng như: thu phí các tính năng nâng cao, không đồng bộ dữ liệu giữa các thiết bị, không hỗ trợ đầy đủ chức năng, ...

Xuất phát từ những nhận định đó, tôi đã chọn đề tài “Phần mềm quản lý chi tiêu cá nhân trên di động” nhằm hỗ trợ người dùng giải quyết các vấn đề trên.

Phần mềm tạm gọi với tên “**MONEY GROUP 4**”

1.3. Đối tượng hướng đến

Với sự bùng nổ của Smartphone, nhóm đối tượng chủ yếu của ứng dụng là các tổ chức/cá nhân có và biết sử dụng smart phone ở mọi độ tuổi, cụ thể:

- Những cá nhân không có nhiều kinh nghiệm cũng như kiến thức về quản lý chi tiêu muốn tìm hiểu và trang bị thêm cho mình kiến thức.
- Những cá nhân/tổ chức quan tâm về quản lý chi tiêu, mong muốn hạn chế số tiền chi tiêu không hợp lý.
- Những cá nhân/tổ chức muốn có một công cụ giúp dễ dàng quản lý mọi nguồn tiền chi/thu của mình.
- Những cá nhân/tổ chức chưa hài lòng về các công cụ hỗ trợ quản lý chi tiêu cá nhân đang sử dụng.

1.4. Môi trường phát triển ứng dụng

- Hệ điều hành phát triển: Microsoft Windows

- Hệ điều hành cài đặt: Android
- Cơ sở dữ liệu đám mây: Firebase
- Công cụ phân tích thiết kế: figma.com
- Công cụ xây dựng ứng dụng: Android Studio, Visual Studio Code, Draw.io
- Ngôn ngữ sử dụng: Dart
- Framework: Flutter

1.5. Kết quả mong đợi

- Học hỏi cách phát triển một ứng dụng trên di động qua quá trình làm đề tài.
- Triển khai được một sản phẩm hoàn thiện, có ích cho người dùng.
- Hiểu được và áp dụng quy trình phát triển phần mềm.

1.6. Khảo sát hiện trạng

Thực tế hiện nay, Việt Nam đã có nhiều ứng dụng quản lý chi tiêu quen thuộc có thể kể đến như: Money Lover, Sổ thu chi, Spendee, MISA Money Keeper, Ta cùng đi qua một số thông tin chung về các ứng dụng này.

Ứng dụng Chức năng	Money lover	Sổ thu chi	Sổ thu chi misa	Nhóm
Thêm, chỉnh sửa thu chi	X	X	X	Khả quan
Tra cứu thu chi	X	X	X	Khả quan
Báo cáo theo tuần , tháng,năm	X	X	X	Khả quan
Liên kết theo dõi tài khoản ngân hàng	X			Không khả quan
Sử dụng trên nhiều thiết bị	X		X	Khả quan
Nhắc nhở các khoản thu chi định kì	X	X	X	Khả quan
Chuyển đổi tiền tệ	X		X	Khả quan

Quét hóa đơn	X			Không khả quan
Lập hạn mức chi tiêu	X		X	Khả quan
Thiết lập số dư ban đầu	X	X	X	Khả quan
Sổ tiết kiệm	X	X	X	Không khả quan
Sổ nợ	X			Không khả quan
Xuất dữ liệu	X	X	X	Khả quan
Danh sách mua sắm, du lịch			X	Không khả quan
Tính lãi vay, thuế TNCN			X	Không khả quan

1.7. Quy trình thực hiện các công việc chính

Hiện nay có rất nhiều quy trình phát triển phần mềm khác nhau. Tuy nhiên theo yêu cầu của đề tài, nhóm đã sử dụng mô hình thác nước cải tiến. Mô hình trên bao gồm các trình tự: xác định yêu cầu, phân tích, thiết kế, cài đặt, kiểm thử, bảo trì. Trong đó, kết quả của giai đoạn trước là cơ sở đầu vào của giai đoạn sau. Vì vậy, nếu như có lỗi xảy ra, nhóm có thể quay lui để sửa lỗi và tối ưu phần mềm trong khi tiến độ hiện tại vẫn được duy trì.

Cụ thể các trình tự phát triển phần mềm của nhóm như sau:

- Xác định yêu cầu: Khảo sát yêu cầu người dùng, lập ra bảng các yêu cầu và quy định cụ thể cho phần mềm.
- Phân tích: Phân loại các yêu cầu và lập sơ đồ Use-case
- Thiết kế: Mô tả các thành phần của phần mềm một cách rõ ràng, gồm các bước:
 - Thiết kế hệ thống, kiến trúc, các đối tượng.

- Thiết kế cơ sở dữ liệu.
- Thiết kế giao diện.
- Cài đặt: Dựa theo những thiết kế và phân tích, tiến hành xây dựng ứng dụng thực tế.
- Kiểm thử: Chạy thực nghiệm và đánh giá, tìm và sửa lỗi.

2. Phân tích, thiết kế hệ thống

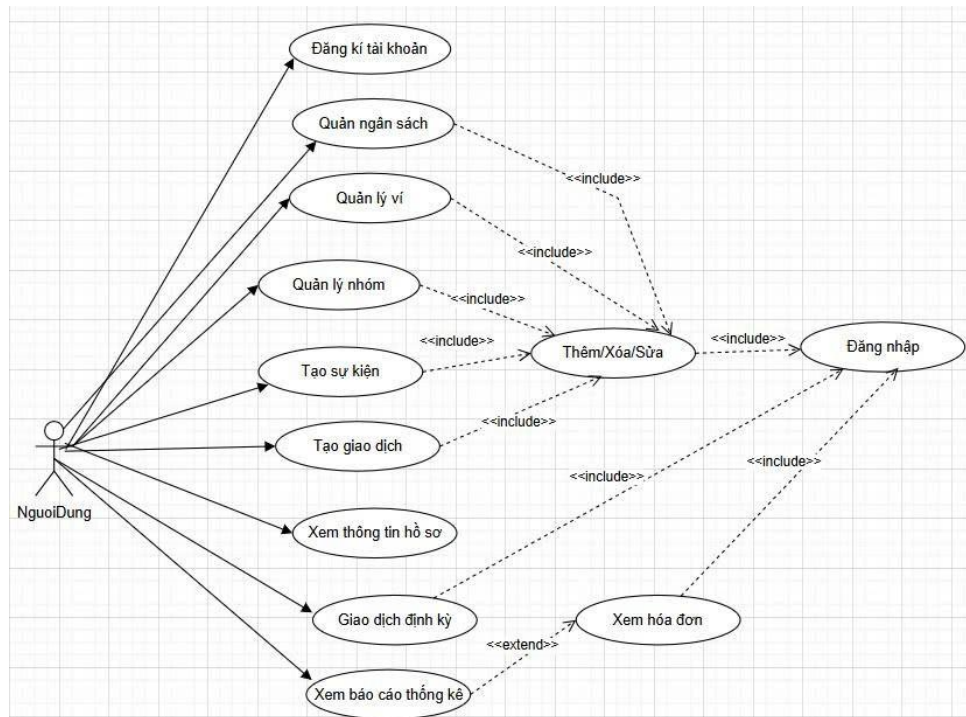
2.1. Xác định và mô hình hóa các yêu cầu phần mềm

2.1.1. Mô hình hóa yêu cầu

a. Danh sách chức năng

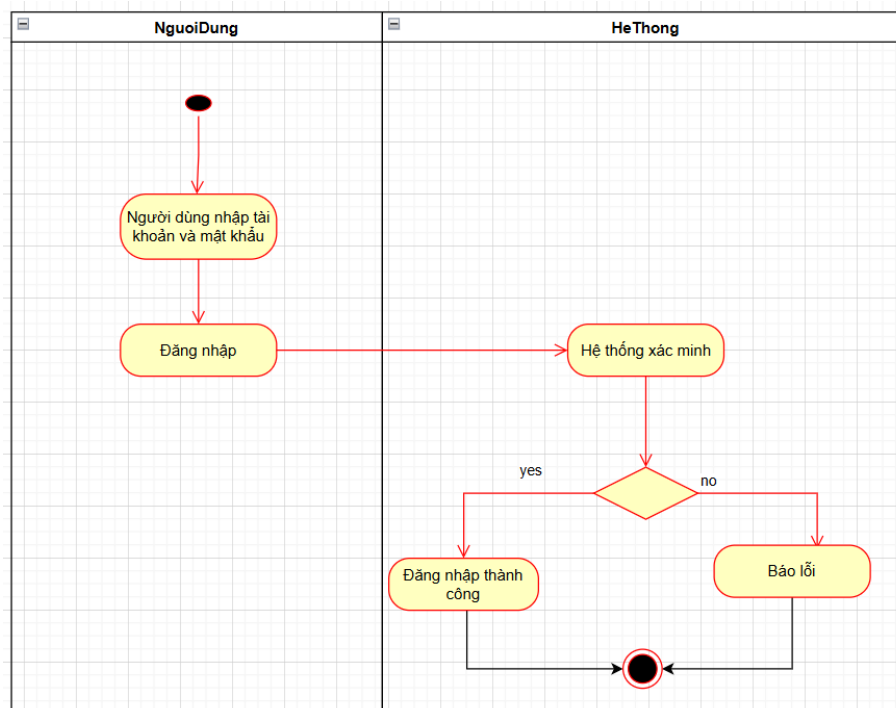
- Đăng ký, đăng nhập
- Quên mật khẩu, thay đổi mật khẩu
- Thêm, sửa, xóa chi tiêu
- Xem thống kê, báo cáo theo các khoảng thời gian nhất định
- Thay đổi thông tin cá nhân
- Thay đổi số tiền quản lý
- Tìm kiếm chi tiêu

b. Lược đồ Use-case tổng quát

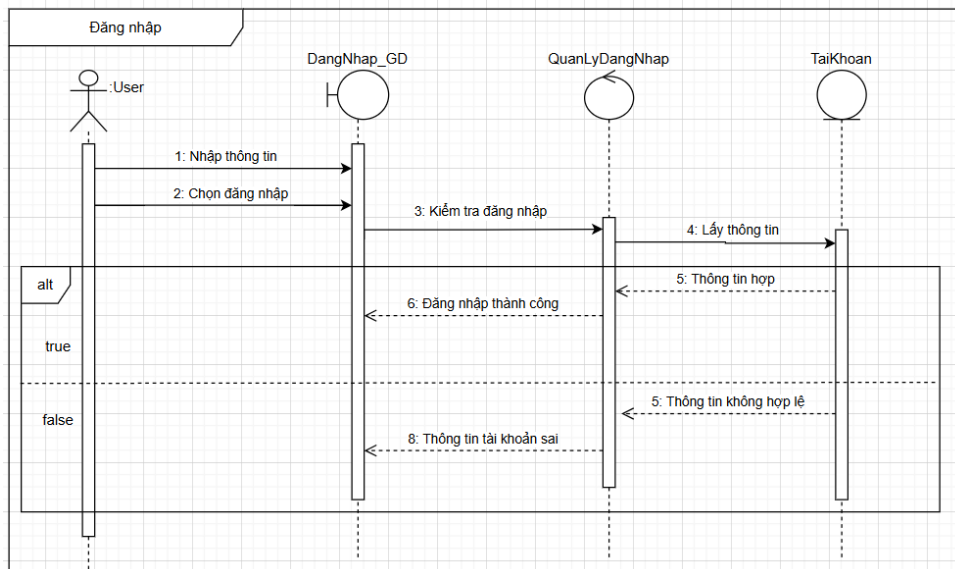


c. Sơ đồ hoạt động và sơ đồ tuần tự

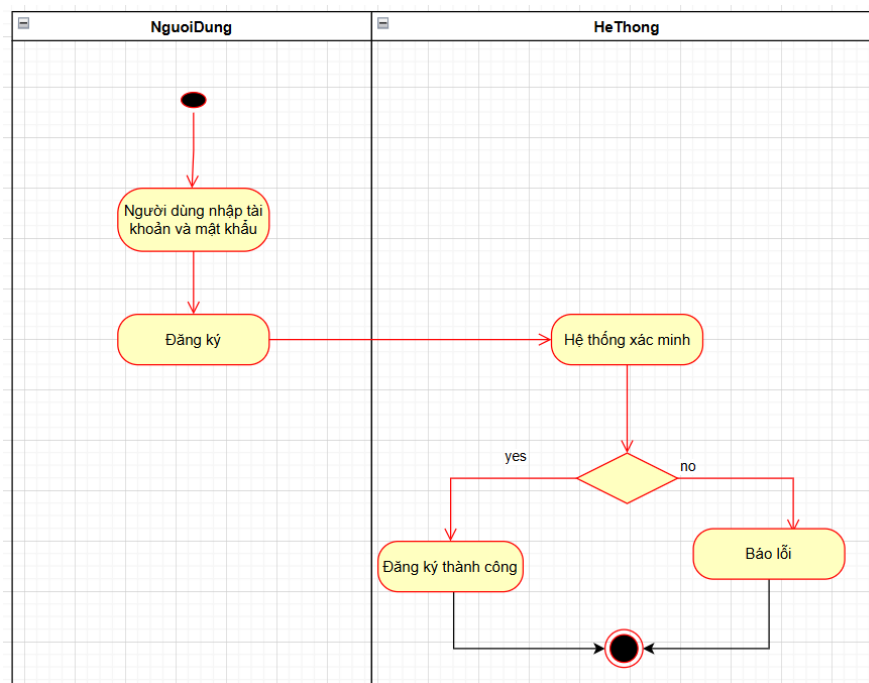
❖ Sơ đồ hoạt động của chức năng Đăng nhập



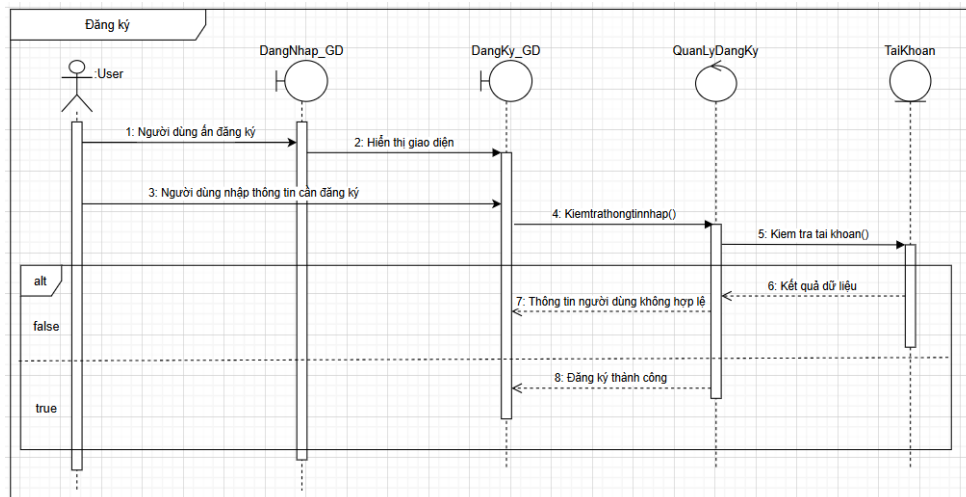
❖ Sơ đồ tuần tự chức năng đăng nhập



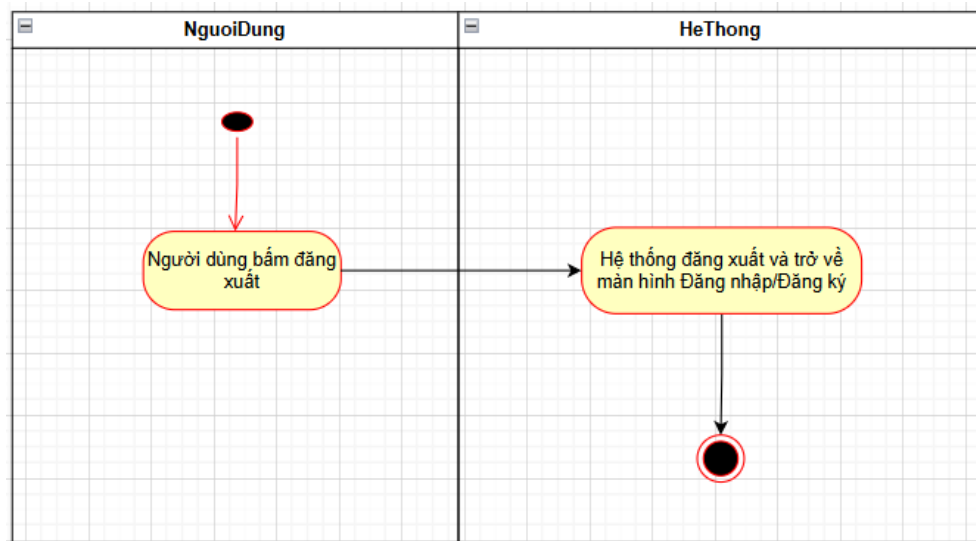
❖ Sơ đồ hoạt động của chức năng đăng ký



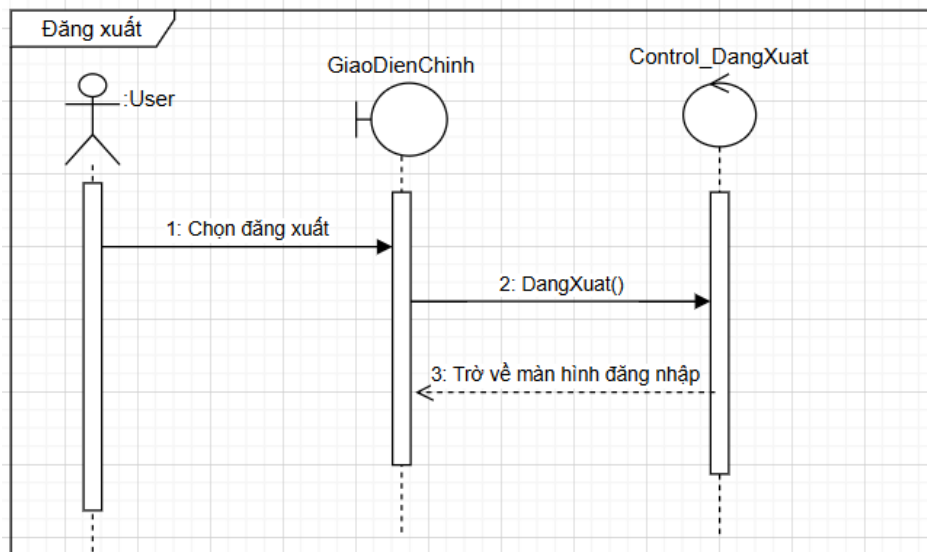
❖ Sơ đồ tuần tự chức năng đăng ký



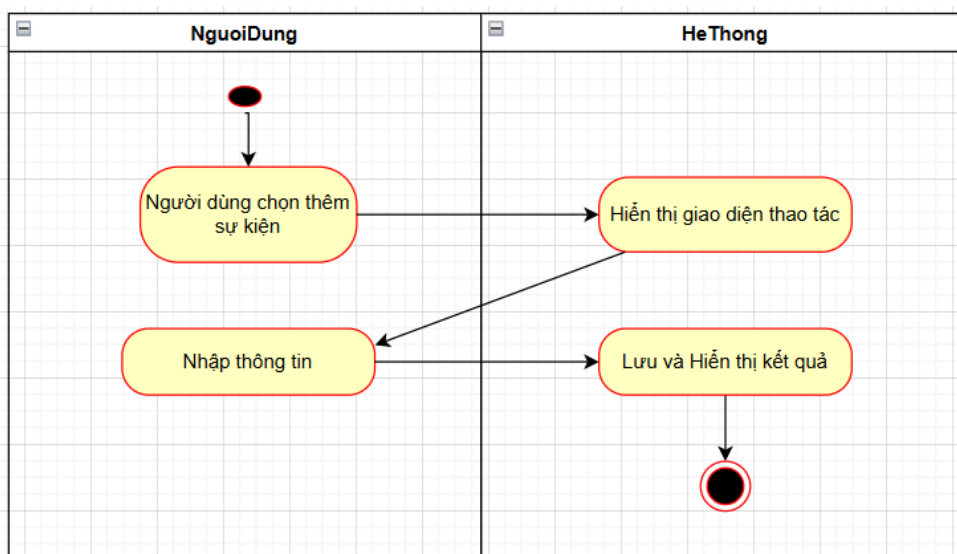
❖ Sơ đồ hoạt động chức năng đăng xuất



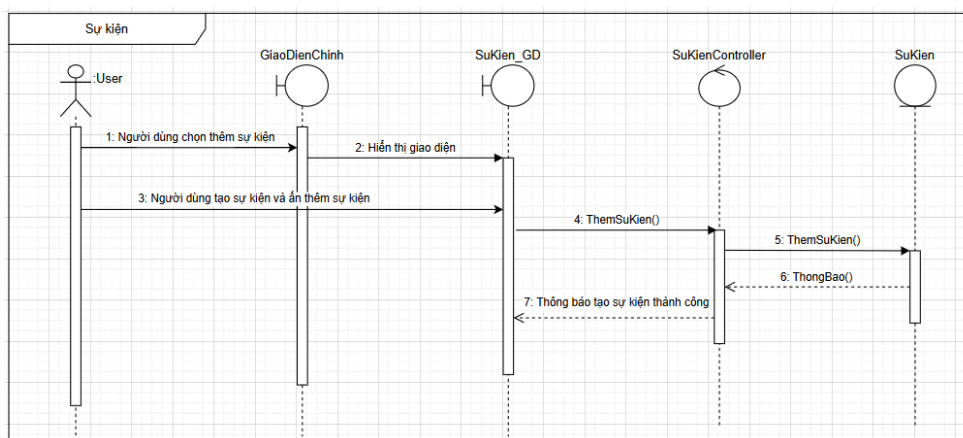
❖ Sơ đồ hoạt động chức năng đăng xuất



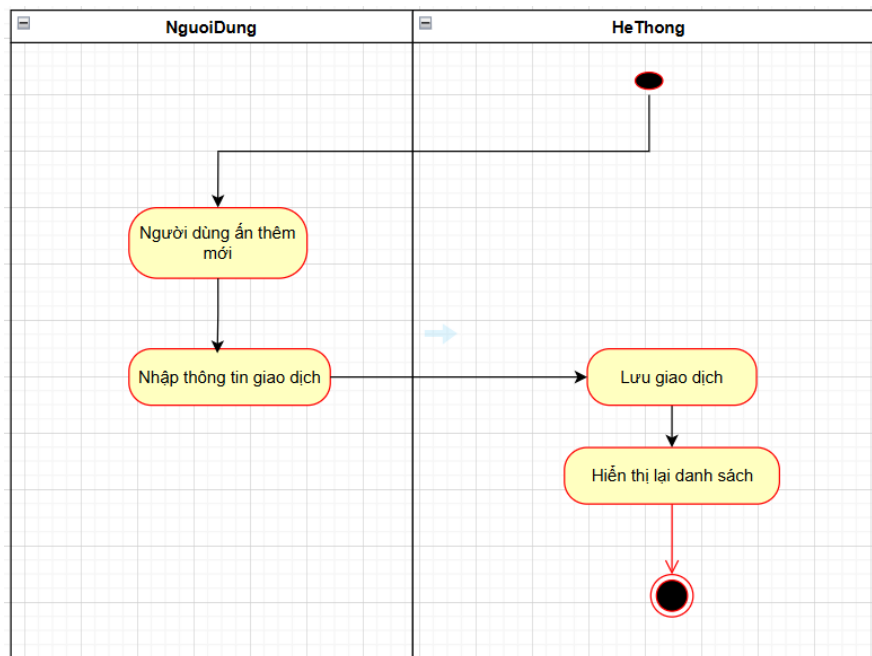
❖ Sơ đồ hoạt động của chức năng tạo sự kiện



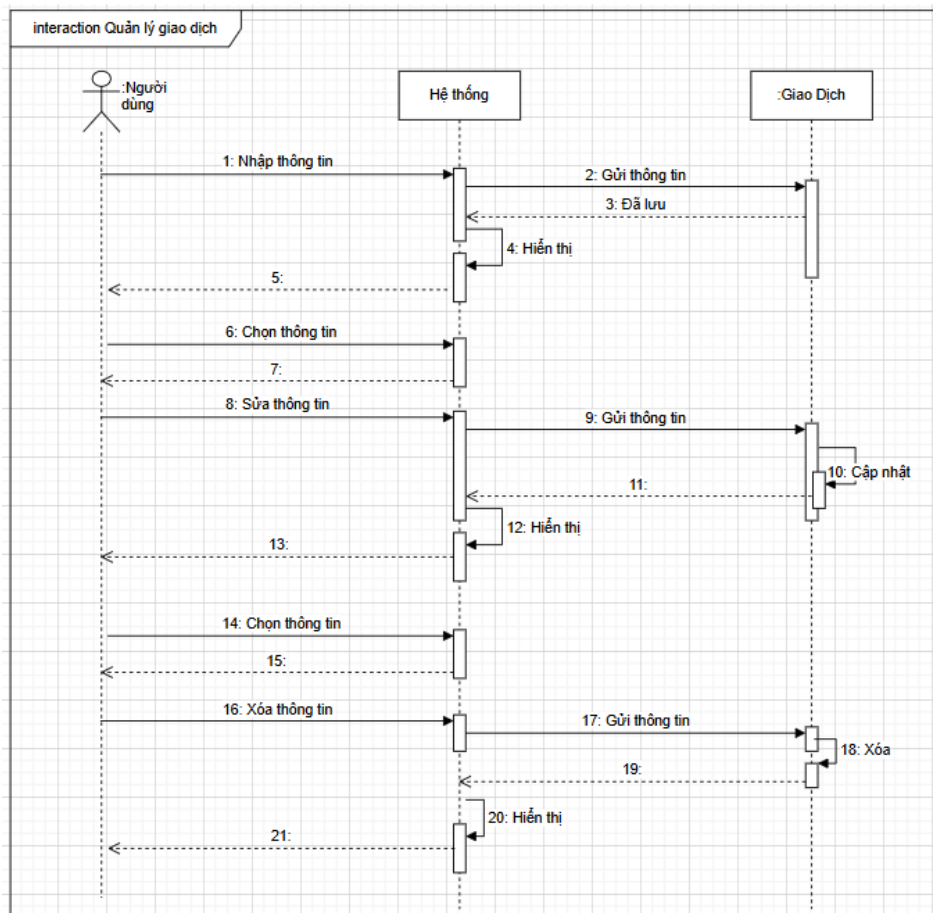
❖ Sơ đồ tuần tự của chức năng tạo sự kiện



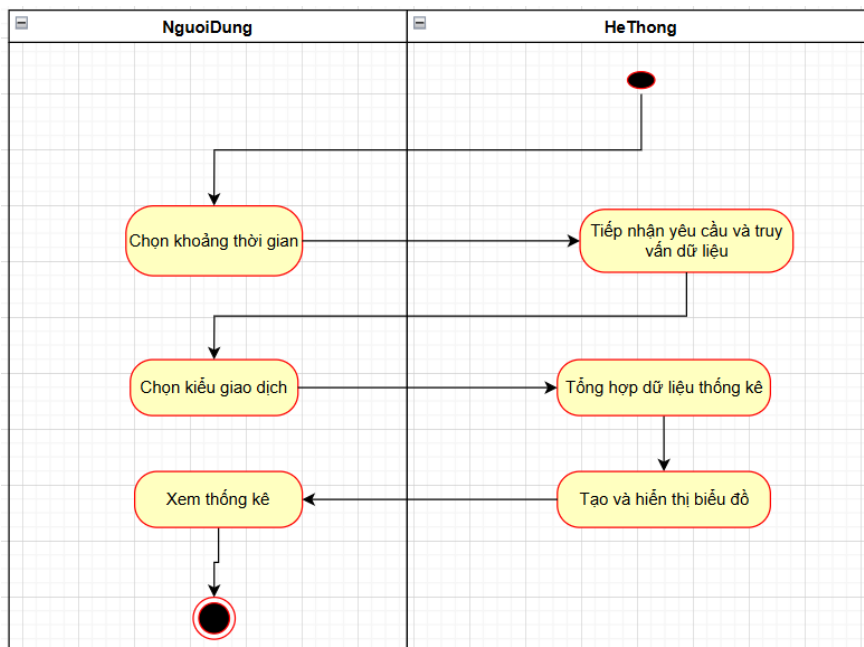
❖ Sơ đồ hoạt động thêm giao dịch



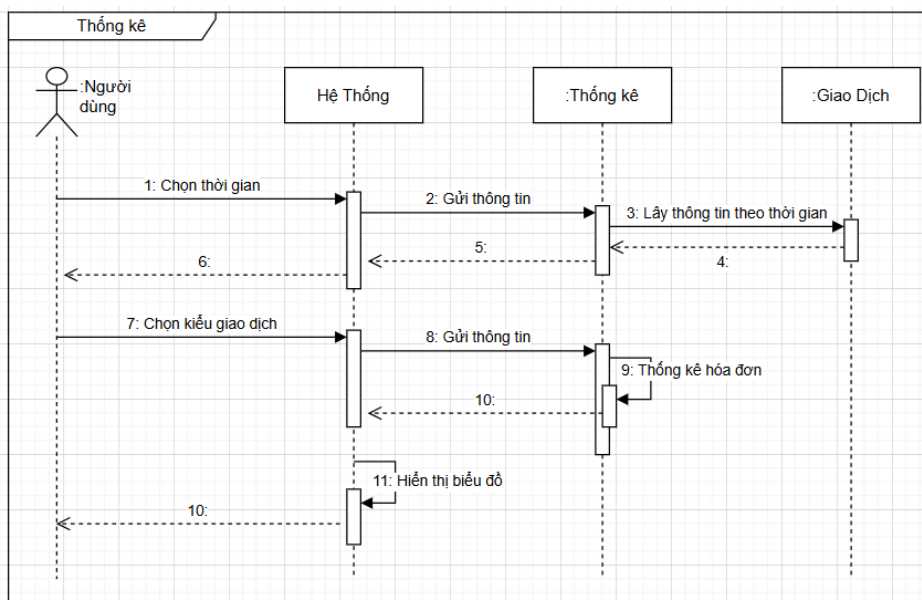
❖ Biểu đồ tuần tự quản lý giao dịch:



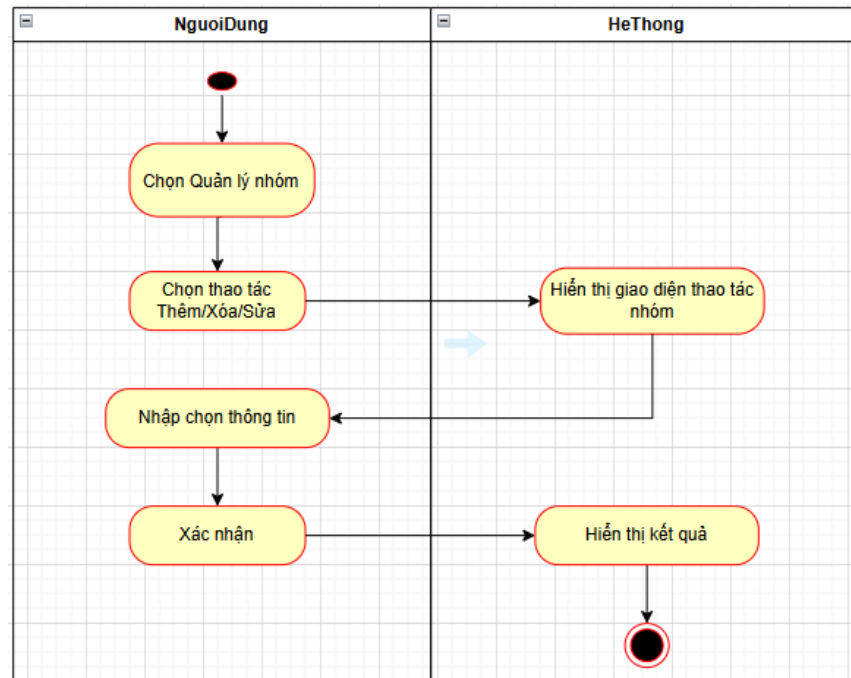
❖ Sơ đồ hoạt động của chức năng thống kê



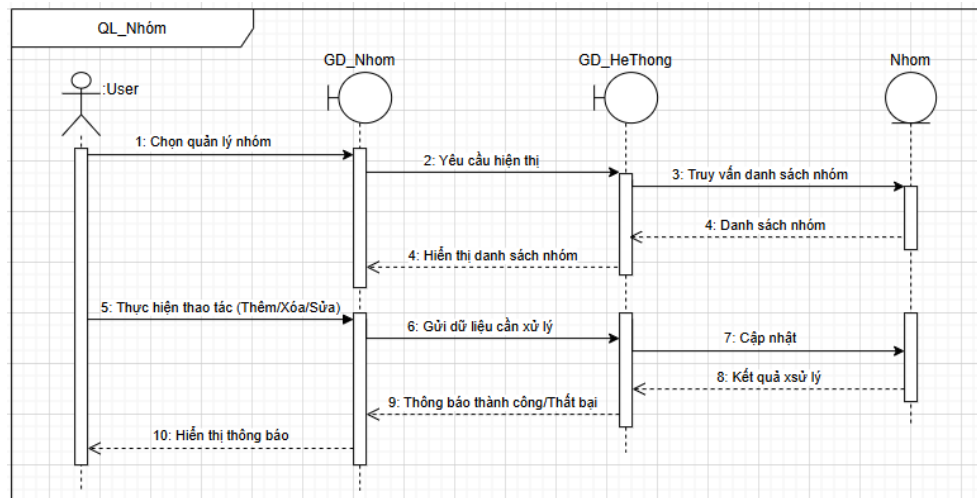
❖ Biểu đồ tuần tự thống kê



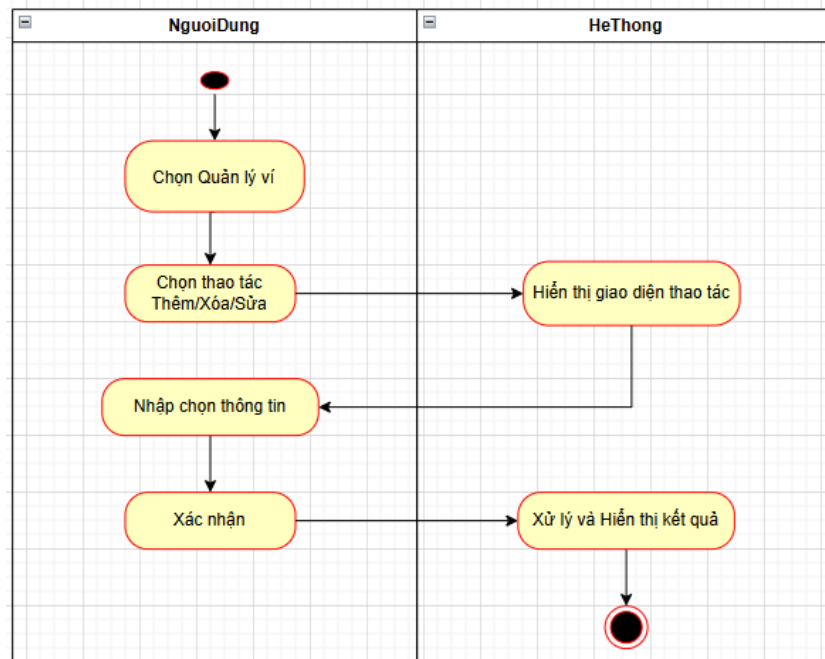
❖ Sơ đồ hoạt động của Quản lý nhóm



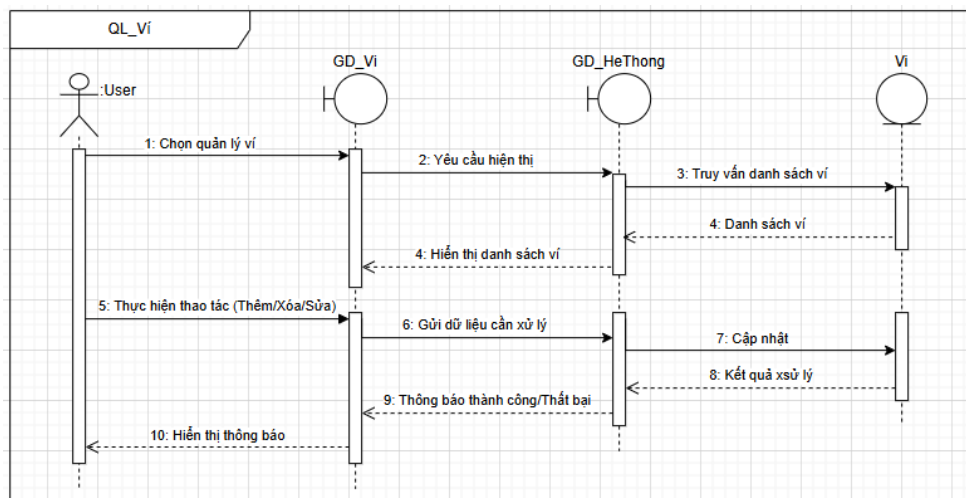
❖ Sơ đồ tuần tự Quản lý nhóm



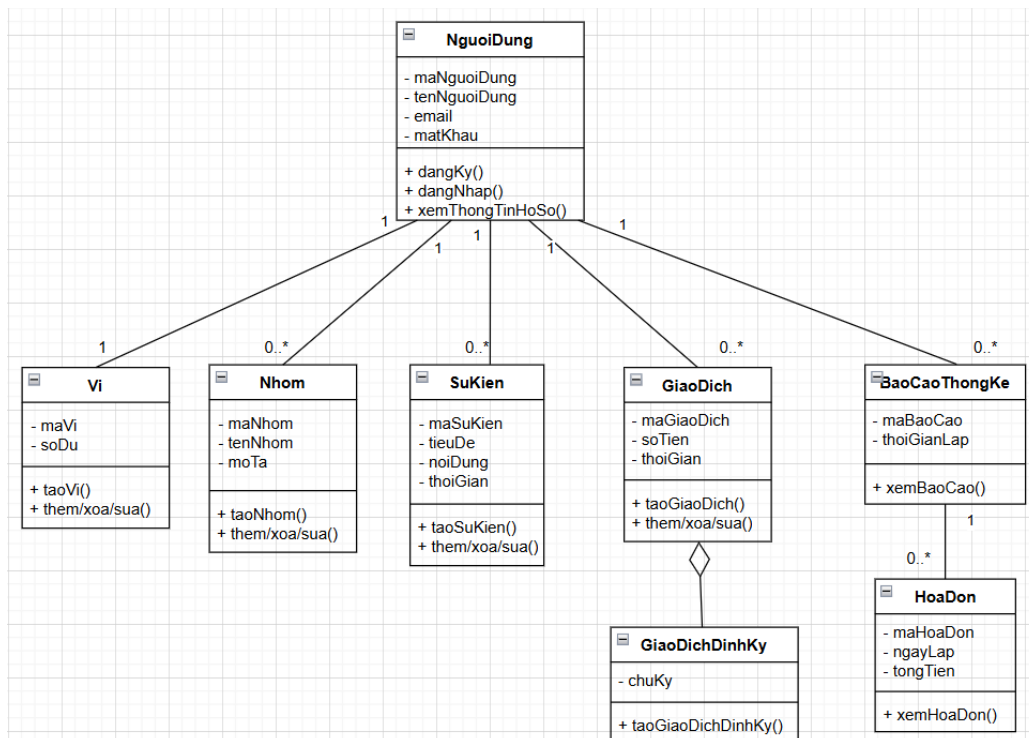
❖ Sơ đồ hoạt động Quản lý ví



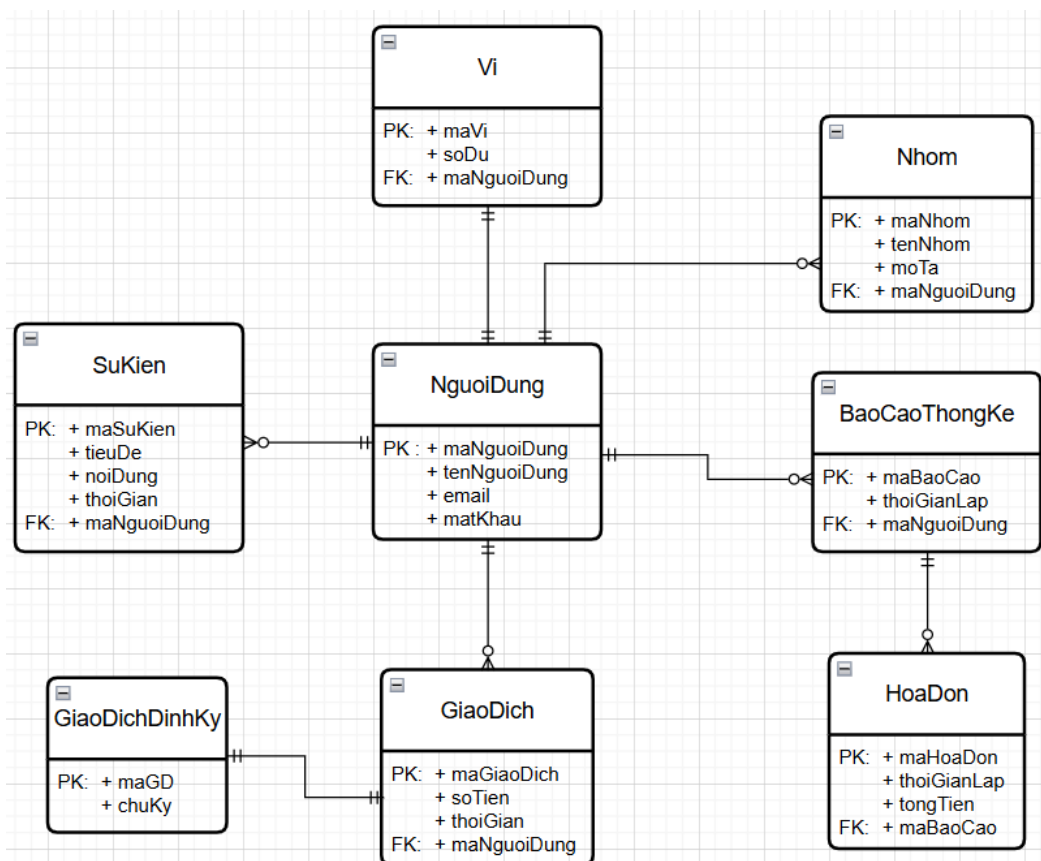
❖ Sơ đồ tuần tự Quản lý ví



d. Sơ đồ class diagram:



e. Mô hình ERD



2.2. Thiết kế hệ thống

2.2.1. Kiến trúc hệ thống

Phần mềm sử dụng kiến trúc bloc:

Bloc giúp dễ dàng tách biệt phần UI và Business logic, giúp code nhanh, dễ test và dễ sử dụng. Mục đích của Pattern này là tách code business logic ra khỏi UI thay vì code gộp chung cả logic và UI vào cùng 1 file, để sau này spec mới có yêu cầu sửa code business logic hay sửa UI sẽ dễ dàng sửa hơn. Code business logic được tách ra đó người ta đặt tên là Bloc (Business Logic Component). Bên cạnh đó, nó còn giúp chúng ta quản lý state của 1 màn hình tốt hơn vì các state sẽ được quản ở Bloc tách biệt với UI. Chính vì vậy, mỗi màn hình trong app flutter chúng ta nên tạo ra 1 bloc để xử lý logic của màn hình đó và quản lý state của cả màn hình đó.

Bloc được coi như bộ não với nhiệm vụ chuyển đổi (convert) các đầu vào là các stream Event thành các stream State ở đầu ra.

Kiến trúc Bloc

Về kiến trúc Bloc trong Flutter, có hai dạng mà bạn thường gặp đó là :

- Xây dựng Bloc với RxDart
- Xây dựng BLoC với Event – State

Nhưng dù dùng kiểu nào đi nữa, thì cấu trúc cũng theo một mô hình như bên dưới:



2.2.2. Mô tả các thành phần trong hệ thống

STT	Thành phần	Diễn giải
1	UI (User Interface)	Là những phần của Ứng dụng để hiển thị với người dùng.
2	BloC	Luôn lắng nghe các sự kiện pass qua nó, ví dụ luôn lắng nghe data pass qua stream.

3	Repository	Dùng để fetching data từ Data sources. Đầu ra của lớp này sẽ là đầu vào của khối Bloc, khi đó data sẽ được đặt trong các Stream
4	Data Sources	Là khối cung cấp data cho ứng dụng như network, sqflite, shared_preferences

5. Hướng dẫn sử dụng phần mềm

5.1. Bắt đầu với hệ thống

Sau khi đọc tập tài liệu này, bạn có khả năng:

- Truy cập vào hệ thống;
- Đăng nhập hoặc đăng xuất khỏi hệ thống;
- Truy cập những tính năng của hệ thống;

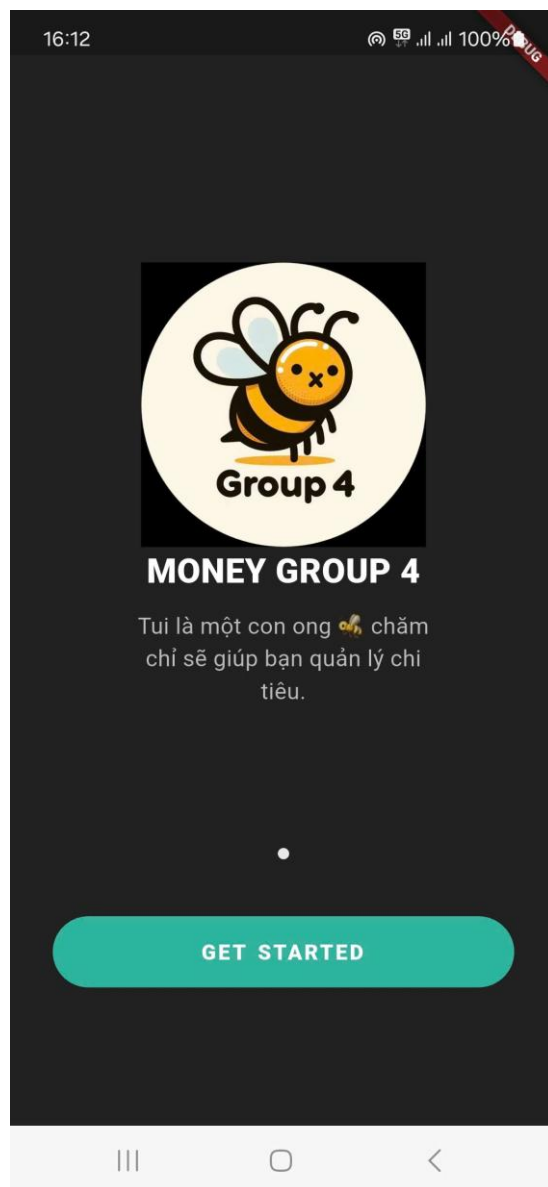
2.3. Thiết kế giao diện

2.3.1. Danh sách các màn hình

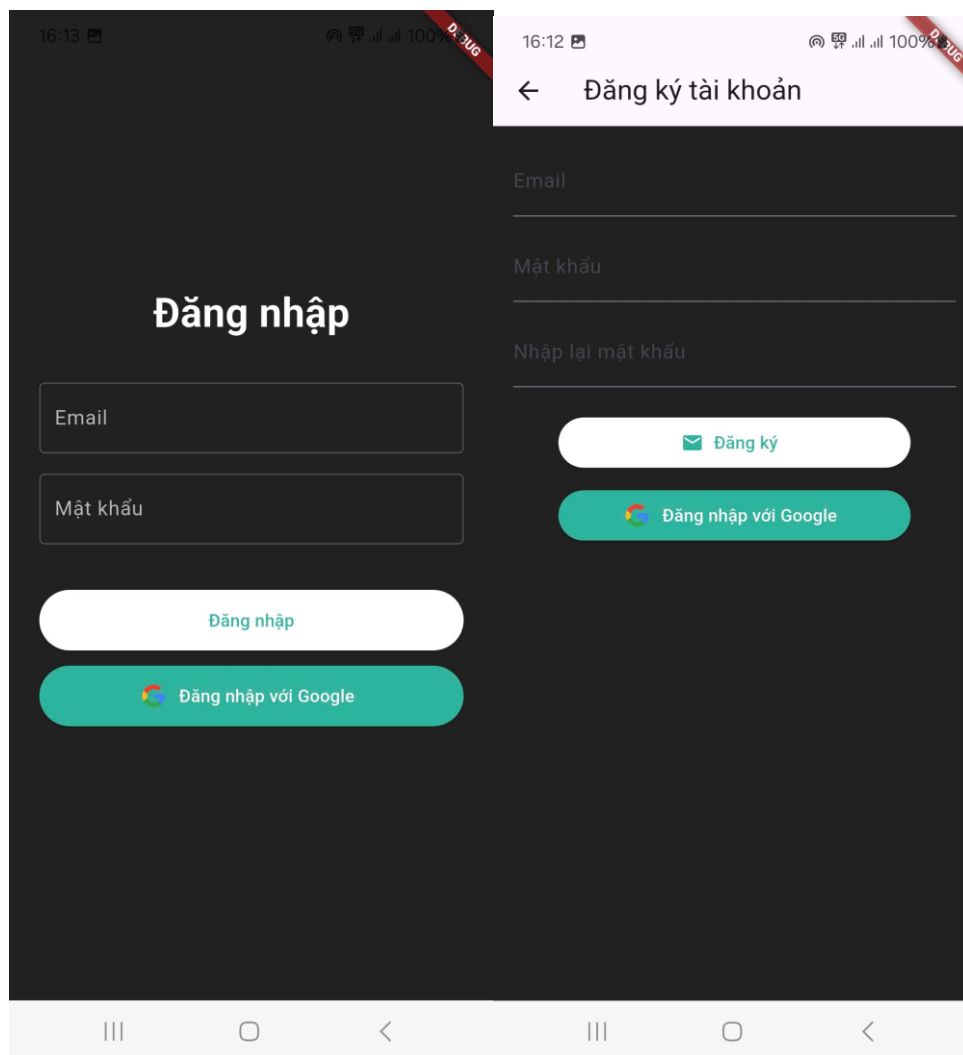
- Màn hình giới thiệu
- Màn hình đăng nhập/đăng ký
- Màn hình sự kiện
- Màn hình hồ sơ
- Màn hình trang chủ
- Màn hình ví, tạo/sửa ví
- Màn hình quản lý nhóm
- Màn hình thêm/sửa/xóa giao dịch
- Màn hình báo cáo
- Màn hình thống kê chi tiêu
- Màn hình thêm ngân sách

2.3.2. Mô tả các màn hình

2.3.2.1. Màn hình giới thiệu



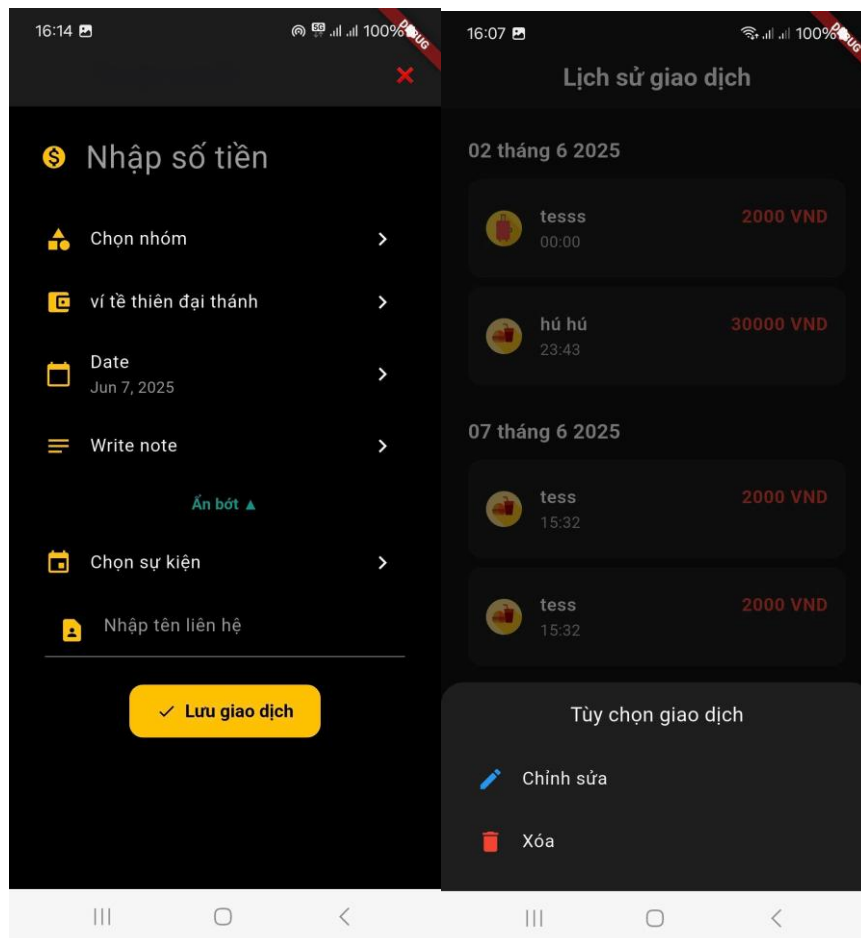
2.3.2.2. Màn hình đăng nhập/dăng ký



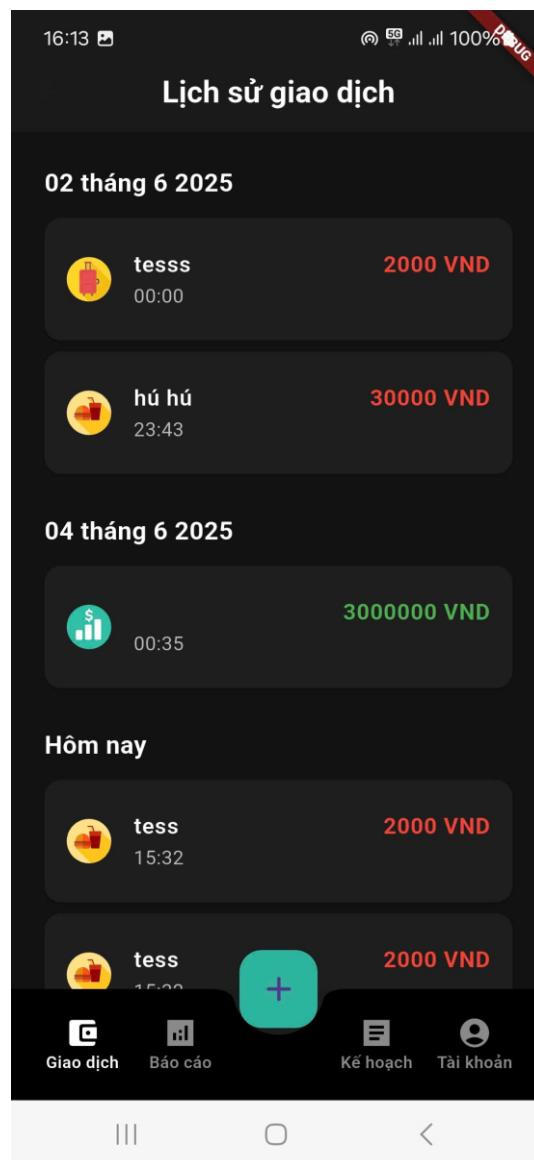
2.3.2.3. Màn hình báo cáo thống kê



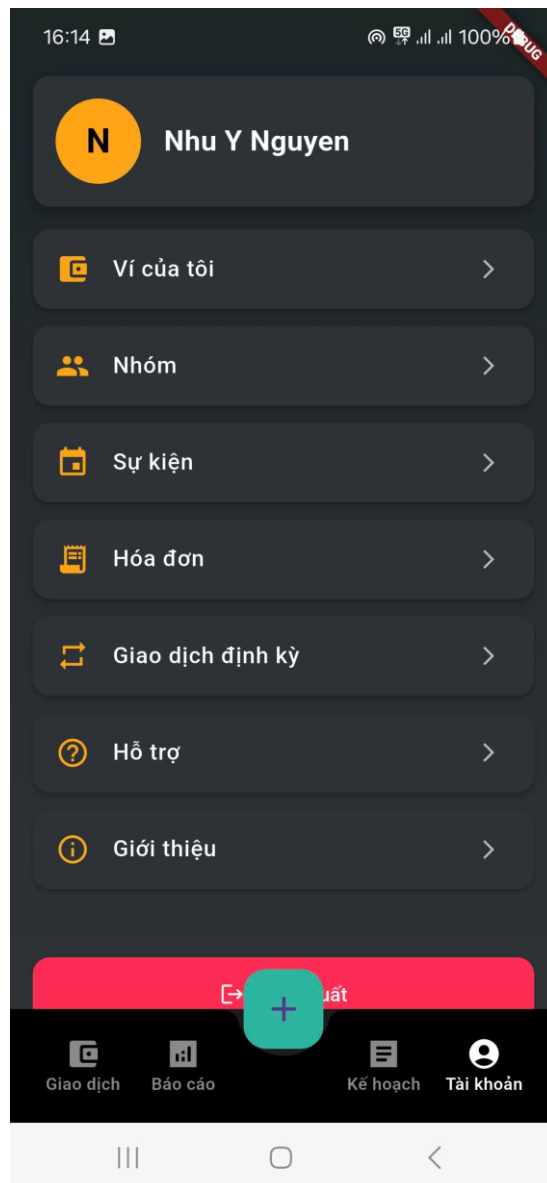
2.3.2.4. Màn hình Thêm/Sửa/Xóa giao dịch



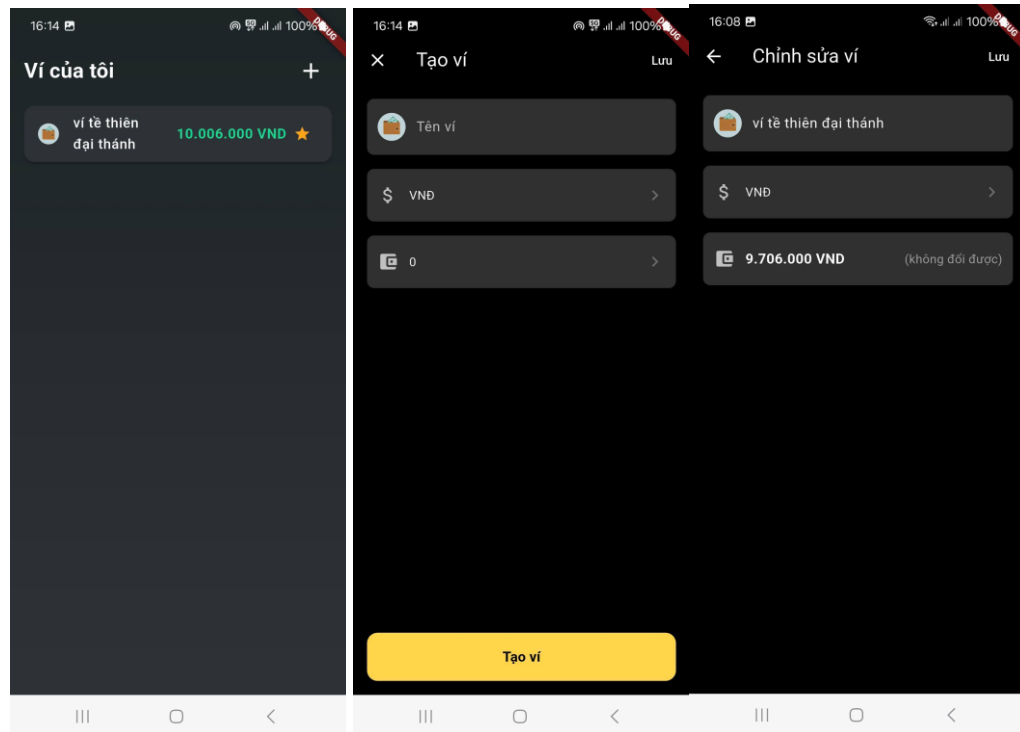
2.3.2.5. Màn hình trang chủ



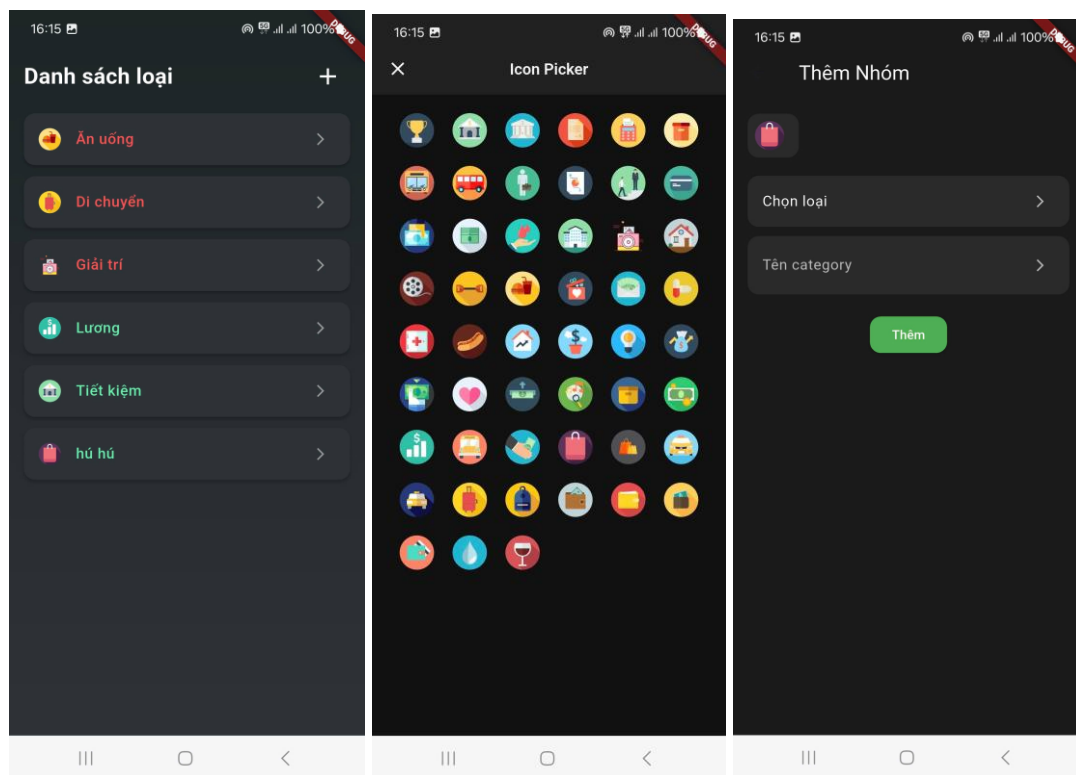
2.3.2.6. Màn hình Hồ sơ cá nhân



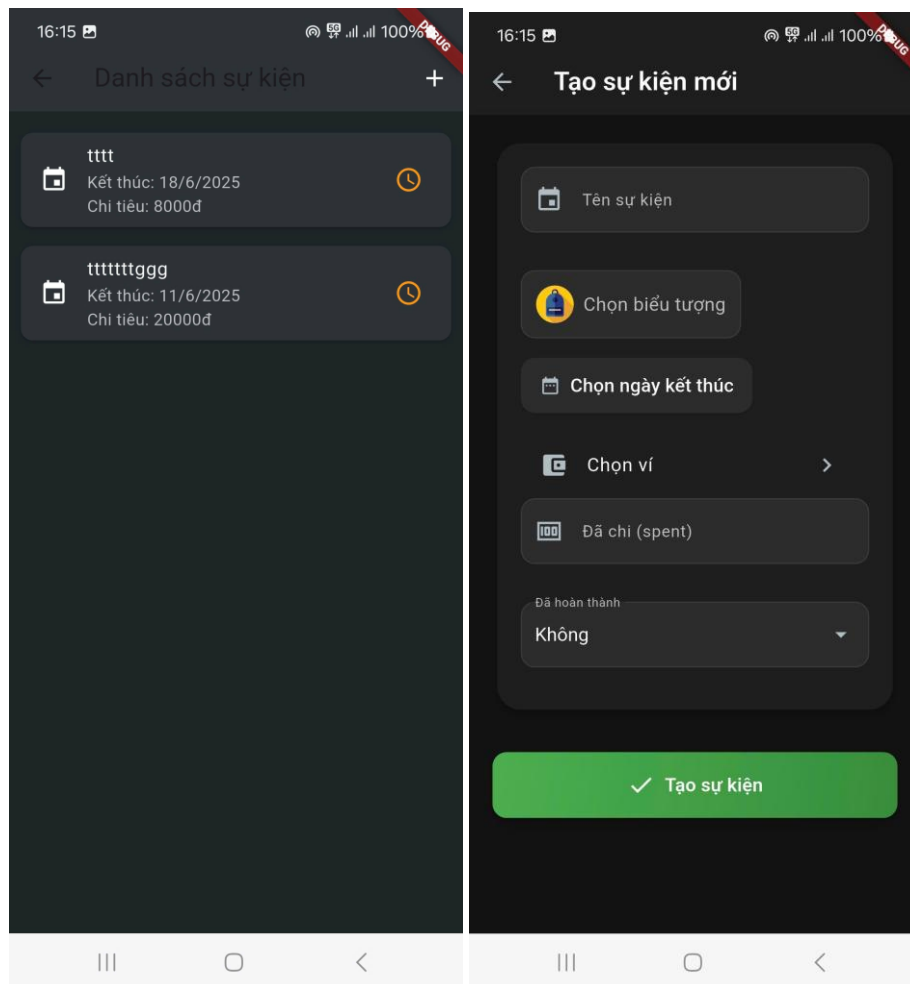
2.3.2.7. Màn hình Ví,tạo Ví và sửa Ví



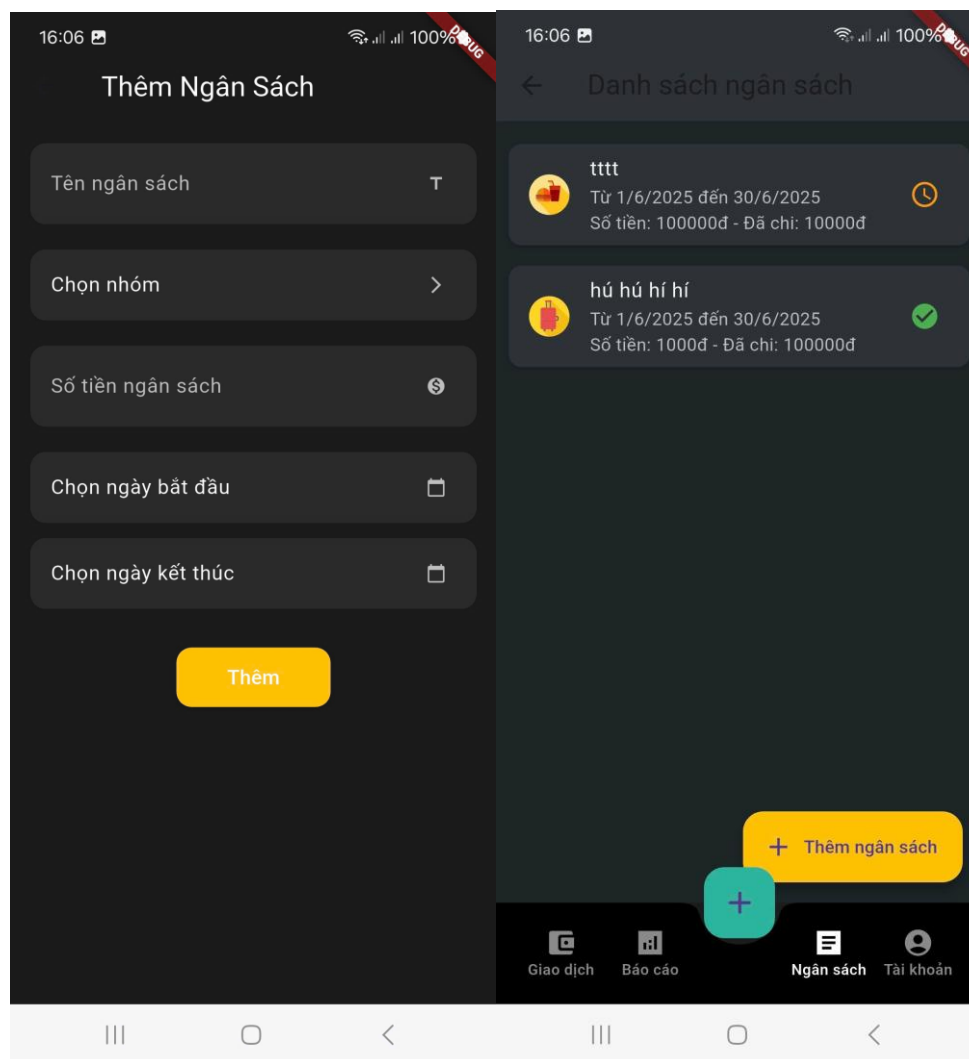
2.3.2.8. Màn hình Quản lý nhóm



2.3.2.9. Màn hình Sự kiện



2.3.2.10. Màn hình tạo ngân sách



5.2. Đăng nhập, đăng ký và đăng xuất khỏi hệ thống

Khi bạn thấy màn hình đăng nhập của ứng dụng, bạn hãy nhập **tên đăng nhập (email)** và **mật khẩu (password)** để đăng nhập vào ứng dụng.

Nếu đăng nhập thất bại, bạn sẽ thấy thông báo tương ứng từ hệ thống. Bạn hãy thử lại.

Tại màn hình đăng nhập bạn nhấn vào **Đăng ký ngay** để di chuyển đến màn hình đăng ký. Bạn cần điền đầy đủ thông tin cần thiết và nhấn **Đăng ký** để tiến hành tạo tài khoản trong hệ thống. Sau khi đăng ký thành công bạn vui lòng kiểm tra email và xác thực tài khoản để có thể truy cập vào hệ thống.

Nếu đăng ký thất bại, bạn sẽ thấy thông báo tương ứng từ hệ thống. Bạn hãy thử đăng ký lại.

Nếu bạn muốn đổi tài khoản quản lý bạn có thể vào **Tài khoản** và chọn **Đăng xuất**

5.3. Chuyển tiếp giữa các thành phần bên trong ứng dụng

Khi vừa truy cập vào ứng dụng bạn sẽ thấy có màn hình chính và phía cuối màn hình sẽ có một thanh bottom tab. Bottom tab này dùng để di chuyển giữa các tính năng bên trong ứng dụng.

Trang chủ là nơi sẽ hiển thị số dư hàng tháng của bạn. Bạn có thể xem chi tiết mỗi tháng bằng cách chọn các tháng trên thanh tab bar phí trên.

Trang lịch là nơi bạn có thể xem được chi tiết chi tiêu từng ngày của bạn bằng cách chọn ngày mà bạn muốn xem trên lịch

Thêm chi tiêu là nơi bạn có thể thêm vào chi tiêu của mình

Báo cáo là nơi bạn có thể thống kê chi tiêu theo tuần, tháng, năm của mình theo biểu đồ cột hoặc tròn.

Tài khoản là nơi bạn có thể quản lý thông tin cá nhân và các cài đặt khác của ứng dụng

5.4. Thêm chi tiêu

Trong màn hình chính bạn nhấn vào biểu tượng dấu +. Màn hình thêm chi tiêu sẽ hiển thị. Bạn tiến hành thêm những thông tin cần thiết vào bên trong và tiến hành lưu chi tiêu. Chi tiêu sẽ được lưu trên hệ thống.

5.5. Sửa chi tiêu

Chọn chi tiêu màn bạn muốn sửa. Tiếp theo chọn biểu tượng cây bút để tiến hành sửa chi tiêu. Nhập vào nội dung mà bạn muốn thay đổi sau đó nhấn **lưu** để lưu lại thay đổi.

CHƯƠNG 4. KẾT LUẬN

Trong quá trình thực hiện đồ án "Xây dựng ứng dụng Quản lý chi tiêu", em đã có cơ hội nghiên cứu và áp dụng kiến thức về lập trình Flutter cùng các thư viện, công cụ liên quan như Firebase, SQLite và các widget trong Flutter để xây dựng một ứng dụng thực tiễn, có khả năng hỗ trợ người dùng trong việc quản lý tài chính cá nhân một cách hiệu quả.

Thông qua việc thiết kế giao diện, xây dựng chức năng, lưu trữ dữ liệu và xử lý tương tác người dùng, em đã nắm vững quy trình phát triển ứng dụng di động đa nền tảng, cũng như tích lũy thêm kỹ năng xử lý vấn đề, phân tích yêu cầu và tối ưu hóa trải nghiệm người dùng. Bên cạnh đó, đồ án cũng giúp em hiểu rõ hơn về cách tổ chức mã nguồn, sử dụng các mô hình thiết kế phù hợp và đảm bảo khả năng mở rộng trong tương lai.

Tuy vẫn còn những hạn chế và thiếu sót, nhưng đồ án là tiền đề quan trọng giúp em tự tin hơn trong việc nghiên cứu và phát triển các sản phẩm phần mềm thực tế sau này. Em xin chân thành cảm ơn thầy Đỗ Gia Bảo đã tận tình hướng dẫn và truyền đạt những kiến thức quý báu trong suốt quá trình thực hiện đồ án.

❖ Hướng phát triển

Mặc dù ứng dụng quản lý chi tiêu đã đáp ứng được các chức năng cơ bản như thêm, sửa, tìm kiếm và thống kê chi tiêu, nhưng trong tương lai em mong muốn phát triển thêm các tính năng nâng cao như:

- **Tích hợp AI** để phân tích thói quen chi tiêu và đề xuất ngân sách hợp lý.
- **Hỗ trợ đa ngôn ngữ**, giúp người dùng ở nhiều khu vực có thể sử dụng ứng dụng dễ dàng hơn.
- **Đồng bộ dữ liệu đa thiết bị**, bằng cách tích hợp các nền tảng lưu trữ đám mây như Google Drive, iCloud.
- **Thiết kế báo cáo trực quan**, sử dụng biểu đồ và dashboard để người dùng theo dõi chi tiêu hiệu quả hơn.
- **Tích hợp ví điện tử hoặc kết nối ngân hàng**, giúp người dùng theo dõi biến động tài chính theo thời gian thực.

Với những định hướng trên, em mong muốn ứng dụng không chỉ dừng lại ở việc theo dõi chi tiêu mà còn trở thành **trợ lý tài chính cá nhân thông minh** cho người dùng.

TÀI LIỆU THAM KHẢO

- [1]. Viblo: <https://viblo.asia/>
- [2]. Bài giảng: <https://sites.google.com/view/success-key/1%E1%BA%ADp-tr%C3%ACnh-di-%C4%91%E1%BB%99ng/flutter-dart?authuser=0>
- [3]. Google Developers. (2024). *Flutter Documentation*. Truy cập từ: <https://docs.flutter.dev>
- [4]. Dart.dev. (2024). *Dart Programming Language Documentation*. Truy cập từ: <https://dart.dev/docs>
- [5]. Firebase. (2024). *Get started with Firebase for Flutter*. Truy cập từ: <https://firebase.google.com/docs/flutter/setup?hl=vi&platform=ios>
- [6]. SQLite Consortium. (2023). *SQLite Documentation*. Truy cập từ: <https://www.sqlite.org/docs.html>
- [7]. Reso Coder. (2022). *Flutter Clean Architecture — TDD, Bloc, and Provider*. Truy cập từ <https://resocoder.com>
- [8]. Nguyễn Văn Hiếu. (2022). *Lập trình Flutter từ cơ bản đến nâng cao*. NXB Khoa học và Kỹ thuật.
- [9]. Đặng Kim Tiến. (2023). *Lập trình Dart và Flutter: Hướng đối tượng, Widgets, State Management*. NXB Hồng Đức.
- [10]. Medium.com. (2023). *State Management in Flutter using Provider & Bloc*. Truy cập từ: <https://medium.com/flutter>
- [11]. Pub.dev. (2024). *Packages for Flutter: provider, get, sqflite...* Truy cập từ <https://pub.dev>